TWIST: A Tool to Accurately and Efficiently Analyze Traveling Wave Solutions in Reaction-Diffusion Systems

Vincent Lovero*

March 4, 2026

Contents

*Mathematics Department, University of California, Davis, USA

Part I
TWIST Reference Manual

1 What is TWIST

Traveling Wave Investigation Software Tool (TWIST) is a command line tool for the computation, continuation, and bifurcation analysis of traveling wave solutions in reaction diffusion systems. TWIST was originally designed to handle high-dimensional, electrophysiological models of cardiac tissue. Therefore, TWIST is capable of computing dispersion curves and spectra of traveling wave solutions with near machine precision accuracy. Existing software that is capable of generating dispersion curves was not designed for complicated, high-dimensional systems, nor were they designed for fast and accurate computation of spectra.

The goal of TWIST was not only to be capable of performing the necessary computations, but to also be *fast* and *easy* to use. TWIST uses algorithms outlined in the later sections that are ideal for large, ill-conditioned systems such as those found in models of cardiac tissue. TWIST is capable of automating almost every step in the computation of dispersion curves and aims to not require a user to have any knowledge of the inner workings of the software.

TWIST offers the following capabilities:

- automatically generate and compile a model from simple text
- compute traveling wave solutions (and an initial wave)
- continue traveling wave solutions through parameter space
- perform bifurcation analysis on families of traveling wave solutions
- simulation traveling wave computed solutions in time
- plot traveling wave solutions, disperion curves, and kymographs of simulations

TWIST does all of the above quickly *and* accurately.

2 Building and Installing TWIST

Building TWIST is meant to be a straightforward process. Depending on the speed of your computer, it can take from 5-30 minutes to complete, and will output a *lot* of text. However it only has to be done *once*. To build TWIST, the following dependencies are needed:

1. **git**
   - **MacOS:** Can be installed with `xcode-select --install` or `brew install git`
   - **Ubuntu:** Can be installed with `sudo apt install git`
2. **cmake** (version 3.24 or later)
   - **MacOS:** Can be installed with `brew install cmake`
   - **Ubuntu:** Can be installed with `sudo apt install cmake`
3. **C++20 compiler** (tested on Clang, GCC, and Apple-Clang)
   - **MacOS:** Can be installed with `xcode-select --install` or `brew install llvm` or `brew install gcc`
   - **Ubuntu:** Can be installed with `sudo apt install build-essential`
4. **gfortran compiler**

- **MacOS:** Can be installed with `brew install gfortran`
- **Ubuntu:** Can be installed with `sudo apt install gfortran`

5. **OpenMP**
   - **MacOS:** Can be installed with `brew install libomp`
   - **Ubuntu:** Should come with gcc after installing C++ compiler
6. **Python** (version 3.9 or later)
   - **MacOS:** Can be installed using the instructions at `https://www.python.org/`.
   - **Ubuntu:** Can be installed using the instructions at `https://www.python.org/`.

To build TWIST, a Python script is provided to perform all of the necessary checks and steps for the user to successfully compile the program. To compile the program, run the command `python3 build.py --optimized` for the optimized version of TWIST. If you do not wish to run the optimized version, use the `--debug` flag instead. This will instead build a debugging version of TWIST that is useful for code development. For all the options available in the build script, pass the `-h/--help` flag.

The build script will download, build, and install all static dependencies:

1. OpenBLAS : used for high performance dense linear algebra
2. UMFPack : used for sparse linear solves
3. libfmt : used for fast and pretty printouts
4. simdjson : used to parse spec file
5. HDF5 : used for serialization of computed data
6. SymEngine : used to generate symbolic derivatives
7. ordered map : used to make dependent auxillary functions possible
8. argparse : used to create the CLI
9. indicators : used for progress bars
10. magic enum : used to make TWIST's help printout nicer and more understable
11. gmp : dependency of SymEngine
12. FFTW : used for FFT based time integrator (IIF2)
13. Caliper : used for (optional) profiling of TWIST

To test if the build worked, run the command in the terminal `twist --help` to get the help readout for the program.

NOTE: The build script installs the TWIST binary to the path `/usr/local/bin/` by default which requires super user (sudo) permissions and is not compatible with Windows. If permissions are not available or if using Windows, pass the `--local-install` flag to the build script, e.g. `python3 build.py --optimized --local-install`. The TWIST program will be installed under the `./build/bin` directory instead. To run twist, use `./build/bin/twist` instead or add `/your/full/path/to/build/bin` to your `PATH`.

3 Using TWIST

This section provides a *brief* overview of some of the core capabilites of TWIST and how to use TWIST. For a more complete description of TWIST's features that are not mentioned in this manual, use the `twist --help` for the complete printout of every command and option. If only the help menu for a specific command is wanted use `twist <command name> --help`.

3.1 Model Specification Files

The way that TWIST interprets a mathematical model is through the JSON file format (see `https://www.json.org/` for the official JSON documentation). Because TWIST is designed for a general reaction-diffusion system, the user is required to input information about the reaction dynamics and the diffusion coefficients.

Suppose you want to study the Fitzhugh-Nagumo equations. The first steps to create the model specification file (spec file) would be to name the model and define the reaction equations. To do this you would define the `name` and `system` entries in the JSON file.

```
"name": "fhn",
"system": {
    "v": "f(v) - w",
    "w": "eps * (v - gamma * w)"
}
```

Next the system parameters must be specified. System parameters are values that can be changed during continuation and will not be treated as permanently constant. This is done with the `params` entry. For the Fitzhugh-Nagumo equations, the parameters are `alpha`, `gamma`, and `eps`

```
"params": [
    "alpha",
    "gamma",
    "eps"
]
```

The Fitzhugh-Nagumo equations also contain what is commonly referred to as an "auxillary" equation, $f(v) = v(v-\alpha)(1-v)$. This can be defined using the the `aux` entry in your spec file,

```
"aux": {
    "f(v)": "v * (v - alpha) * (1 - v)"
}
```

Note that for auxillary equations, the key entry (here it is `"f(v)"`), *must* be an *exact* match to what is written in the system equations. If it is not, then it is likely that an ambiguous error message about something differentiation not being implemented will be printed out and the program will terminate.

Next the diffusion coefficients must be specified for each variable (v and w). This is done using the `diffusion` entry in the spec file. For non-diffusive variables (diffusion coefficient is zero), if a diffusion coefficient is not specified, it is assumed to be zero. For the Fitzhugh-Nagumo equations it is common for the w state to be non-diffusive, therefore the diffusion coefficients can be specified in the following way,

```
"diffusion": {
    "v": 1.0
}
```

Lastly, some starting values need to be specified for the system. These are needed for generating an initial traveling wave. You will need to specify a starting spatial period, rest state for the reaction dynamics (crude approximation is fine), and "default" parameter values. For the parameter values, you are able to define multiple parameter sets using the `parameter_sets` entry. Different parameter sets can be used with TWIST's program keyword argument `--parameter-set <name>`, however, if no parameter set is specified, TWIST assumes a parameter set with the name "default" to exist in the spec file. For the Fitzhugh-Nagumo equations, this parameter set can be defined in the following way:

```
"parameter_sets": {
    "default": {
        "alpha": 0.25,
        "gamma": 5.0,
        "eps": 0.003
    }
}
```

The remaining required initial values can be defined with

```
"spatial_period": 200.0,
"rest_state": [ 0, 0 ]
```

Here, `spatial_period` is the default or starting circumference of the periodic domain, and `rest_state` is a guess of the values of a fixed point to the reaction dynamics. The rest state does *not* have to be exact. If the guess is too poor and leads to too much activity before the stimulus, the initial guess can be refined useing the `-r/--refine-rest-state` flag during the later described `initialize` command. Putting all of the above listing snippets into a single JSON file yield the following spec file that defines the Fitzhugh-Nagumo equations.

```
{
    "name": "fhn",
    "system": {
        "v": "f(v) - w",
        "w": "eps * (v - gamma * w)"
    },
    "params": [
        "alpha",
        "gamma",
        "eps"
    ],
    "aux": {
        "f(v)": "v * (v - alpha) * (1 - v)"
    },
    "diffusion": {
        "v": 1.0
    },
    "spatial_period": 200.0,
    "rest_state": [ 0, 0 ],
    "parameter_sets": {
        "default": {
            "alpha": 0.25,
            "gamma": 5.0,
            "eps": 0.003
        }
    }
}
```

Some notes about what else can go into the spec file if desired:

- all standard math functions can be used in system/auxillary equations. TWIST Recognizes the following math functions: `abs`, `ceil`, `floor`, `trunc`, `sin`, `cos`, `tan`, `sinh`, `cosh`, `tanh`, `fmax`, `fmin`, `pow`, `exp`, `sqrt`, `tgamma`, `lgamma`, `atan2`, `log`, `erf`, `erfc`, `asin`, `acos`, `atan`, `asinh`, `acosh`, `atanh`. Note that the `cbrt` function is not in this list. Use `pow(..., 1/3)` in place of `cbrt(...)`.

- piecewise functions can be defined as well. The syntax for a piecewise definition would be

```
"pw_var_or_aux": "Piecewise((expression1, condition1), (
    expression2, condition2), (last_expression, True))"
```

  Note: the last expression *must* have the condition set to `True`. For a more concrete example defining an auxillary piecewise function in a spec file, consider the piecewise linear function found in a simplified version [13] of the Fitzhugh-Nagumo equations

$$f(v) = \begin{cases} v & v < \alpha \\ v - 1 & \text{otherwise} \end{cases}$$

```
"f(v)": "Piecewise((v, v < alpha), (v - 1, True))"
```

- constants (assumed to never change) can be defined using the `constants` entry. If γ was constant, example usage would be

```
"constants": {
    "gamma": 5.0
}
```

- Auxillary functions can depend on each other. But, they *must* be defined in a sequential order. Meaning the following is *invalid*

```
"aux": {
    "aux1": "1 + aux2",
    "aux2": "exp(-5)"
}
```

  because `aux1` is dependent on `aux2` but it wasn't defined until after `aux1` is defined.

- Coordinate transforms can be done on individual states. Suppose you want to transform

$$v \mapsto (v + 80)/120$$

  . This can be done by defining the transform in the spec file in the following way:

```
"transforms": {
    "v": [ 80.0, 120.0 ]
}
```

  This feature is available for two reasons: to make plotting various states more visually appealing, and for numerical stability. If one of the state variables is much larger in magnitude, rounding errors could prevent convergence for small solver tolerances.

A more involved system is the Beeler-Reuter (BR) model. The BR model can be defined using a procedure similar to the one described above. The spec file for the BR model could look something like the following:

```
{
    "name": "br",
    "system": {
        "V": "-(i_K1 + i_x1 + i_Na + i_s) / Cm",
        "Ca": "-r_Ca * i_s + k_up * (Ca_SR - Ca)",
        "x_1": "alpha_x1 * (1 - x_1) - beta_x1 * x_1",
        "m": "alpha_m * (1 - m) - beta_m * m",
        "h": "alpha_h * (1 - h) - beta_h * h",
        "j": "alpha_j * (1 - j) - beta_j * j",
        "d": "alpha_d * (1 - d) - beta_d * d",
        "f": "alpha_f * (1 - f) - beta_f * f"
    },
    "transforms": {
        "V": [
            85.0,
            120.0
        ]
    },
    "aux": {
        "V_Ca": "-82.3 - 13.0287 * log(Ca)",
        "i_s": "g_s * d * f * (V - V_Ca)",
        "i_Na": "(g_Na * m ** 3 * h * j + g_NaC) * (V - V_Na)",
        "i_x1": "A_x1 * x_1 * (exp(0.04 * (V + 77)) - 1) / exp(0.04 * (V +
            35))",
        "i_K1": "A_K1 * (4 * (exp(0.04 * (V + 85)) - 1) / (exp(0.08 * (V +
            53)) + exp(0.04 * (V + 53))) + 0.2 * (V + 23) / (1 - exp(-0.04
            * (V + 23))))",
        "beta_f": "0.0065 * exp(-0.02 * (V + 30)) / (exp(-0.2 * (V + 30))
            + 1)",
        "alpha_f": "0.012 * exp(-0.008 * (V + 28)) / (exp(0.15 * (V + 28))
            + 1)",
        "beta_d": "0.07 * exp(-0.017 * (V + 44)) / (exp(0.05 * (V + 44)) +
            1)",
        "alpha_d": "0.095 * exp(-0.01 * (V - 5)) / (exp(-0.072 * (V - 5))
            + 1)",
        "beta_j": "0.3 / (exp(-0.1 * (V + 32)) + 1)",
        "alpha_j": "0.055 * exp(-0.25 * (V + 78)) / (exp(-0.2 * (V + 78))
            + 1)",
        "beta_h": "1.7 / (exp(-0.082 * (V + 22.5)) + 1)",
        "alpha_h": "0.126 * exp(-0.25 * (V + 77))",
        "beta_m": " (40 * exp(-0.056 * (V + 72)))",
        "alpha_m": " (-(V + 47) / (exp(-0.1 * (V + 47)) - 1))",
        "beta_x1": "0.0013 * exp(-0.06 * (V + 20)) / (exp(-0.04 * (V + 20)
            ) + 1)",
        "alpha_x1": "0.0005 * exp(0.083 * (V + 50)) / (exp(0.057 * (V +
            50)) + 1)"
    },
    "constants": {
        "Cm": 1,
        "r_Ca": 1e-07,
        "Ca_SR": 1e-07,
        "k_up": 0.07,
        "A_K1": 0.35,
```

```
        "A_x1": 0.8,
        "V_Na": 50,
        "g_NaC": 0.003
    },
    "params": [
        "g_Na",
        "g_s"
    ],
    "rest_state": [
        -83.3,
        1.87e-07,
        0.1644,
        0.01,
        0.9814,
        0.9673,
        0.0033,
        0.9884
    ],
    "spatial_period": 25.0,
    "diffusion": { "V": 1e-3 },
    "parameter_sets": {
        "default": {
            "g_s": 0.09,
            "g_Na": 4
        }
    }
}
```

Note that these example (and many others) can be found at `https://www.github.com/vlovero/twist/example_models` or in the `example_models` folder if you have already downloaded TWIST.

3.2 Building Model

Once your spec file has been written, you can use TWIST to build your model. Using the example spec file for the Fitzhugh-Nagumo equations from the previous subsection, to build the model you would use the `build` command in TWIST

```
twist build fhn.json
```

This command will generate a C++ file and compile it for later use with TWIST. The build command only has to be run *once*. However, if the model equations are changed or new parameters are introduced, the model will have to be rebuilt using the `build` command again in order for the changes to take effect. Note that if you have already done many computations using an existing model, then it is rebuilt after changes are made, all of the previously generated data files could be incompatible with the newly built model. Therefore, it is best to change the **"name"** entry in the spec (JSON) file whenever major changes are made. Every file that TWIST generates will go into the `.cache` directory. Model source code will go under `.cache/models/src` and the compiled model will go under `.cache/models/lib`. There is also an option to not compile the generated code and only produce the source code. To do this you would pass the `-s/--source-only` flag. This is useful when you would like to hand code your own equations or pass in your own compilation flags.

3.3 Creating Initial Guess

Once the model has been built, it is ready to start generating traveling waves. To **generate the initial traveling wave**, the `initialize` command is used. For the example Fitzhugh-Nagumo equations we've been using thus far,

you could run the following command the generate an initial traveling wave solution

```
twist initialize fhn.json --time-ode 500 --time-pde 4000
```

Here the user must specify two keyword arguments, `--time-ode`: the integration time for the local dynamics, and `--time-pde`: the total time to run the PDE simulation.

The method that TWIST uses to generate an initial traveling wave solution is by first integrating the local/reaction dynamics with an applied stimulous at a specified point in the time domain. The default is one-tenth of the total time specified by `--time-ode`. This generates an action potential for a single "cell." This action potential can then be used as a starting wave profile for a simulation of the full reaction-diffusion system. This profile is used because the recovery phase is an accurate approximation of the back of a traveling wave solution and will lead to one way propagation around the ring that the reaction-diffusion system will be integrated on.

If the stimulus was not large enough to generate an action potential, the size of the amplitude can be changed using the `--stim-amp` keyword argument. The default is 0.4. The duration of the stimulus can be changed too using the `--stim-dur` keyword argument. The default is 2.

To **view/display the resulting profile** (before and after the PDE simulation), the `-d/--display` flag can be passed. This will result in the following figures: Note that when displaying the computed profiles,

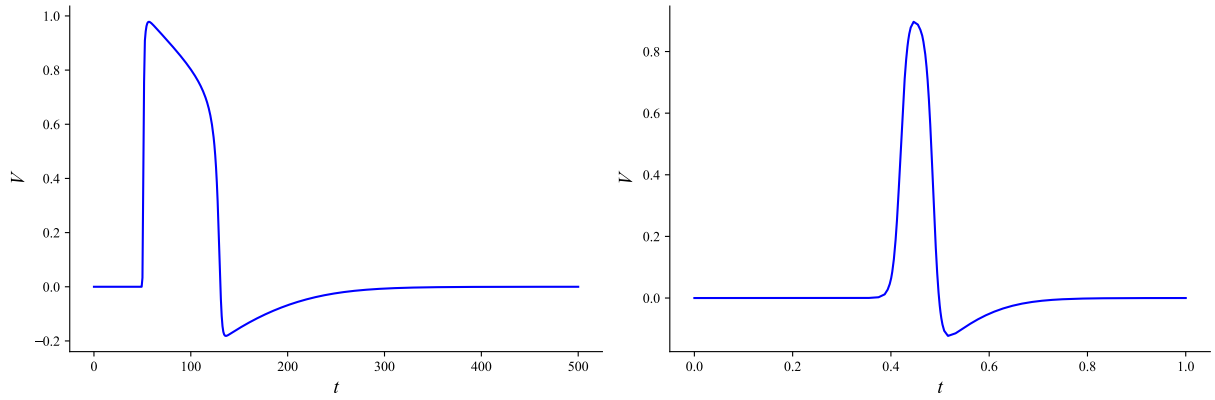


Figure 1: Initialization of the Fitzhugh-Nagumo equations. Plot on the left shows the starting wave profile after time integration has been done on the local dynamics. The plot on the right shows the initial traveling wave solution after time integration of the reaction-diffusion system has been done.

the plotted profiles must be closed in order for the computations to proceed.

3.4 The `solve` Command

The `solve` command has many features built into it and its purpose is to do deep explorations of a single traveling wave solution. Some of the features include

- solve for the traveling wave
- compute the spectrum of the traveling wave (can also compute the essential spectrum)
- simulate the traveling wave
- plot the computed traveling wave, spectrum, and/or space-time plot

To check that the initial traveling wave solution in the previous subsection is a sufficiently accurate guess for the collocation method, we use the solve command.

```
twist solve fhn.json --solve-init --plot-after-solve-init
```

which should produce the following output

```
Using default tolerance of 8.222312e-13
iter  |    ||F||     |    ||dy||    |     damp     |      c       |   |phase|    |  nodes   |
-------------------------------------------------------------------------------------------------
    0 | 3.226500e-02 |      -       |      -       | 2.841263e-01 | 0.000000e+00 |      138 |
    1 | 1.836686e-04 | 2.241644e-01 | 1.000000e+00 | 2.866268e-01 | 0.000000e+00 |      138 |
    2 | 4.367074e-08 | 2.525010e-03 | 1.000000e+00 | 2.866197e-01 | 4.633522e-18 |      138 |
    3 | 5.238346e-14 | 4.782658e-06 | 1.000000e+00 | 2.866196e-01 | 4.684813e-18 |      138 |

Using default tolerance of 2.167155e-13
iter  |    ||F||     |    ||dy||    |     damp     |      c       |   |phase|    |  nodes   |
-------------------------------------------------------------------------------------------------
    0 | 1.066291e-09 |      -       |      -       | 2.866196e-01 | 0.000000e+00 |       37 |
    1 | 5.000358e-16 | 8.253551e-10 | 1.000000e+00 | 2.866196e-01 | 0.000000e+00 |       37 |
solve took 1.499158e-03 sec

Using default tolerance of 1.987299e-13
iter  |    ||F||     |    ||dy||    |     damp     |      c       |   |phase|    |  nodes   |
-------------------------------------------------------------------------------------------------
    0 | 8.225486e-10 |      -       |      -       | 2.866196e-01 | 0.000000e+00 |       34 |
    1 | 4.915675e-16 | 7.015347e-10 | 1.000000e+00 | 2.866196e-01 | 0.000000e+00 |       34 |
solve took 1.139412e-03 sec
Finished with 894 unknowns
```

Note that some of the output is omitted because it did not fit on the page.

The `solve` command solves for the traveling wave solution and adapts the mesh twice to refine the solution to the desired global error tolerance that can optionally be specified with the `--solve-geps` flag. The default tolerance is 10^{-12}. The following metrics are output by the solver (unless the `--solve-quietly` flag):

`iter` : How many iterations/steps the solver has perfomed

`||F||` : The current residual norm for the nonlinear equations

`||dy||` : The norm of the update vector

`damp` : The current damping value being used

`c` : The current estimation of the wave speed

`|phase|` : The current magnitude of the phase condition

`nodes` : The current number of nodes in the mesh

`solve(sec)` : The time it took to factor and solve for the update vector

`rate` : The estimated rate of convergence of the solver

After the solver has finished, it will plot the computed solution profiles. To view and compute the spectrum of this traveling wave solution, pass the `--spectrum` flag to TWIST. Doing so will generate the following figure To **simulate this traveling wave** in time, pass the `--simulate` flag to TWIST. This will result in a simulation of the reaction-diffusion system with the final time value set to be the computed temporal period.

The **essential spectrum** can also be computed using TWIST with the `--essential-spectrum` flag. This flag will tell TWIST to first compute the spectrum then track every eigenvalue as the phase shift, γ, in the essential spectrum boundary conditions,

$$e^{i\gamma}y(0) - y(L) = 0 \tag{1}$$

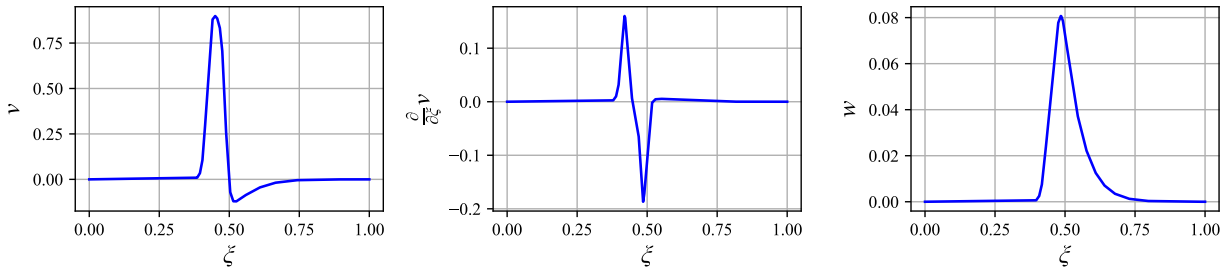
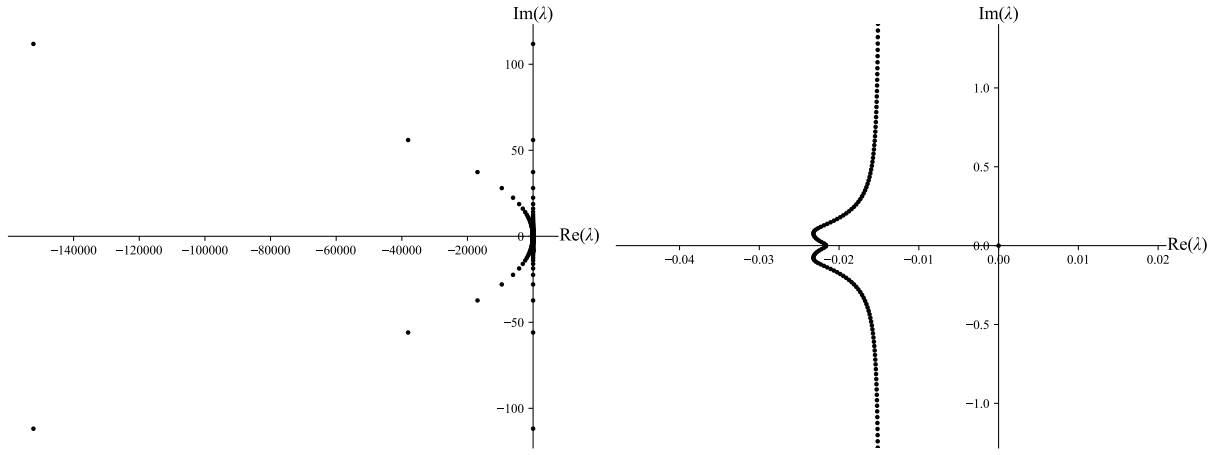


Figure 2: Traveling wave solution computed using the `solve` command.



(a) (b)

Figure 3: Spectrum generated using TWIST with the `--spectrum` flag. (a) the generated figure of the spectrum. (b) zoomed in view of the spectrum in (a). The full command is: `twist solve fhn.json --solve-init --spectrum`

for $\gamma \in [-2\pi, 2\pi]$. For a detailed explanation of the boundary conditions 1, see [16]. This computation is *very* expensive and often unnecessary as only a small handful of eigenvalues are typically followed. To choose which eigenvalues are tracked use the `--essential-spectrum-filter` argument. This argument takes in a mathematical boolean expression to select the eigenvalues. For example only eigenvalues in the right half of the complex plane are desired, then one would pass `"real > 0"`. Conditions can also be "chained" using the "and" and "or" operators, `&&` and `||`. For example if only *positive, real* eigenvalues are desired, one would pass, `"(real > 0) && (imag == 0)"`. All standard binary operators and mathematical functions are supported in these filter expressions.

There are many bells and whistles for the `solve` command. To see the full list of all of the options and settings, run `twist solve --help`.

3.5 Continuing Traveling Wave Solutions

To **generate dispersion curves** using TWIST, the `continuation` command is used. This command is used solely to generate dispersion curves. It requires a few extra keyword arguments than the previous commands. The user must specify not ony the model, but

- The starting direction of the continuation loop. This can be forwards (positive) or backwards (negative). The `-f/--forward` and `-b/--backward` flags set this option.
- Which parameter that TWIST will be performing continuation in. This is set with the `-p/--parameter` keyword argument. The parameter name passed to TWIST must be in the list of parameters defined

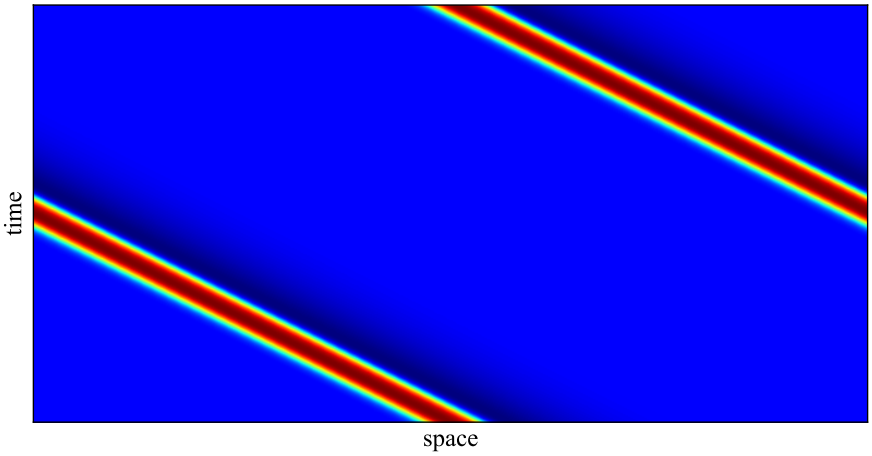


Figure 4: The figure generated after running a simulation in the Fitzhugh-Nagumo equations using the `--simulate` flag. Full command is: `twist solve fhn.json --solve-init --simulate`

in the spec file. Continuation can also be performed in the spatial period. This is done by choosing the parameter, "sps", which stands for spatial period scale. The way that TWIST performs continuation in the spatial period is not through the period, but through the relative spatial period, where the baseline period is the one specified in the spec file.

- The bounds for the parameter, `--parmin` and `--parmax`. If the continuation steps outside of these bounds, the program will finish.
- The initial step size, `--ds`, the minimum step size, `--dsmin`, and maximum step size, `--dsmax`, for the continuation algorithm. For most problems the choice for the min and max step sizes isn't too important, however, for better looking diagrams, a smaller max step size is preferred.

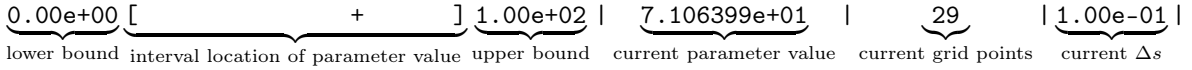
Examples of performing continuation in the Fitzhugh-Nagumo equations are:

```
twist continuation fhn.json --ncol 10 --solve-min-nodes 20 -b -p sps
 --parmin 0 --parmax 1 --ds 0.1 --dsmin 1e-6 --dsmax 0.1 -t example
```

to perform continuation in relative spatial period

```
 twist continuation fhn.json --ncol 10 --solve-min-nodes 20 -f -p eps
--parmin 1e-8 --parmax 4e-3 --ds 0.1 --dsmin 1e-6 --dsmax 0.1 -t example
```

to perform continuation in the ratio of time scales, ε. The continuation commands above will first refine the traveling wave solution using the solver used by the `solve` command. The program will first output the same output as the `solve` command, then a continuation progress monitor will appear.



In the examples above, the number of collocation points is set to ten using `--ncol 10` and the minimum number of grid points in the discretization is set to 20 with `--solve-min-nodes 20`. Setting the minimum number of grid points can be important when passing through regions in parameter space where solutions' gradients are not very large. The number of grid points can become low causing step sizes that are too large for the continuation algorithm to converge. The `-t` keyword argument sets the "tag" for the data that will be generated. It is optional and the default value is set to "latest." Setting the tag is useful when you are generating several dispersion curves using the same continuation parameter. *Note:* It is best to always set

the tag to something unique because if continuation has previously been performed in the parameter that is to be varied with an already existing tag, *the old data will be deleted*.

The resulting dispersion curves and solutions are serialized into a single file using the HDF5 format. The two example dispersion curves will be stored under

```
.cache/continuation_data/fhn-default-sps-example.h5
.cache/continuation_data/fhn-default-eps-example.h5
```

and in general, the file name format is the following

```
<name>-<parameter set>-<parameter>-<tag>.h5
```

To **visualize the data**, use the `visualize` command

```
twist visualize .cache/continuation_data/fhn-default-sps-example.h5
```

which will result in an interactive window shown Figure 5. The `visualize` command is discussed further in a later subsection.

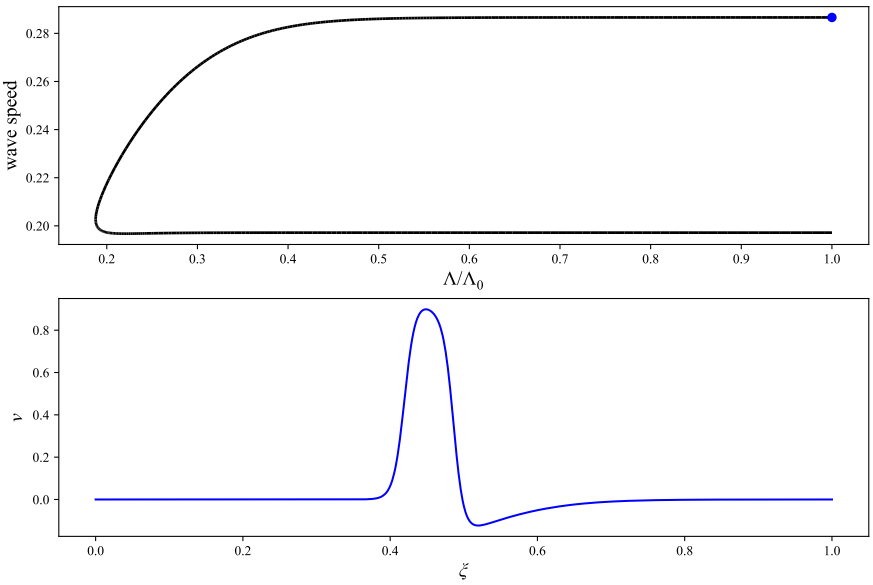


Figure 5: Dispersion curve generated using TWIST for the Fitzhugh-Nagumo equations with relative spatial period as the continuation parameter.

3.6 Bifurcation Analysis

The last step to generate a complete dispersion curve (bifurcation diagram) is to **compute the spectra** of the solutions generated during the continuation process. This is done using the `postprocess` command. The reason that this is a separate step is because it is the most computationally expensive and time consuming. While TWIST uses a novel algorithm to compute the spectrum of a traveling wave much faster than any other software package, it can still consume a lot of time (especially for systems with many state variables).

To compute the spectra for the dispersion curves generated in the previous subsection, pass the `-s/--spectrum` flag to TWIST.

```
twist postprocess .cache/continuation_data/fhn-default-sps-example.h5
       .cache/continuation_data/fhn-default-eps-example.h5 -s
```

To check for any bifurcations (and to compute the bifurcation points), use the `-b/--bifurcations` flag. Using the same `visualize` command as before the dispersion curve will now show the stability (and any bifurcation points). Figure 6 shows the complete dispersion curve.

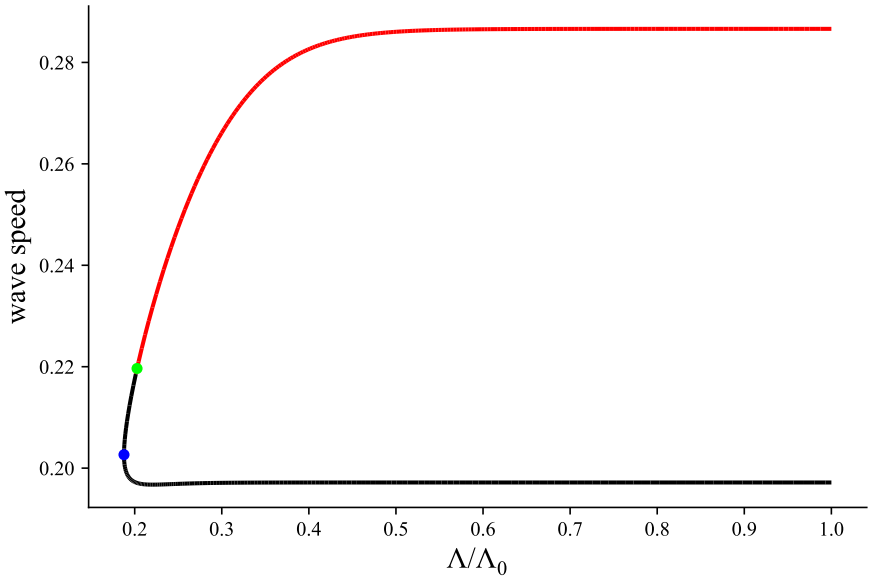


Figure 6: The complete dispersion curve in the Fitzhugh-Nagumo equations using relative spatial period as the continuation parameter.

3.7 Using Existing Data

A useful command line argument found under the `solve` and `continuation` commands is the `--from-data` option. `--from-data` is used to **start the solver at an already computed traveling wave** solution during continuation. This option is useful for generating multiple segments of the same dispersion curve, analyzing temporal dynamics of specific traveling waves, stability analysis of specific traveling waves, etc.

The usage is: `--from-data <path to data> <solution index>`. Here, the path needs to be to the data generated during continuation, and the solution index is which solution from the file you would like the solver to start from. TWIST uses zero-based indexing so 0 would correspond to the first solution in the file. TWIST also supports negative indexing the same way that python does, so -1 would correspond to the last solution, -2 would be the second to last, etc.

An actual example of using the `--from-data` option is in the reduced ten-Tusscher model (TP06). The TP06 model has a very complex dispersion curve structure that requires several steps to compute.

1. Continue the initial traveling wave to a very long spatial period (4 times the default).

```
twist continuation tp06.json --ncol 10 -p sps -f --parmin 0 --parmax 4 --ds 1
                    --dsmin 1e-6 --dsmax 2 -t step1
```

2. Continue the last traveling wave solution from step 1 backwards in spatial period again.

```
twist continuation tp06.json --ncol 10 -p sps -b --parmin 0 --parmax 4 --ds 1
                --dsmin 1e-6 --dsmax 2 -t step2 --from-data
            .cache/continuation_data/tp06-default-sps-step1.h5 -1
```

3. Continue the last traveling wave solution from step 1 backwards in g_{Na}.

```
twist continuation tp06.json --ncol 10 -p GNa -b --parmin 0 --parmax 4 --ds 1
                --dsmin 1e-6 --dsmax 2 -t step3 --from-data
            .cache/continuation_data/tp06-default-sps-step1.h5 -1
```

4. Continue the last traveling wave solution from step 3 backwards in spatial period.

```
twist continuation tp06.json --ncol 10 -p sps -b --parmin 0 --parmax 4 --ds 1
                --dsmin 1e-6 --dsmax 2 -t step3 --from-data
            .cache/continuation_data/tp06-default-GNa-step3.h5 -1
```

Then plottings the two branches from steps 2 and 4 generate the curve(s) in Figure 7.

```
twist visualize .cache/continuation_data/tnnp-default-sps-step2.h5
.cache/continuation_data/tnnp-default-sps-step4.h5 --individual --fixed-solutions
```

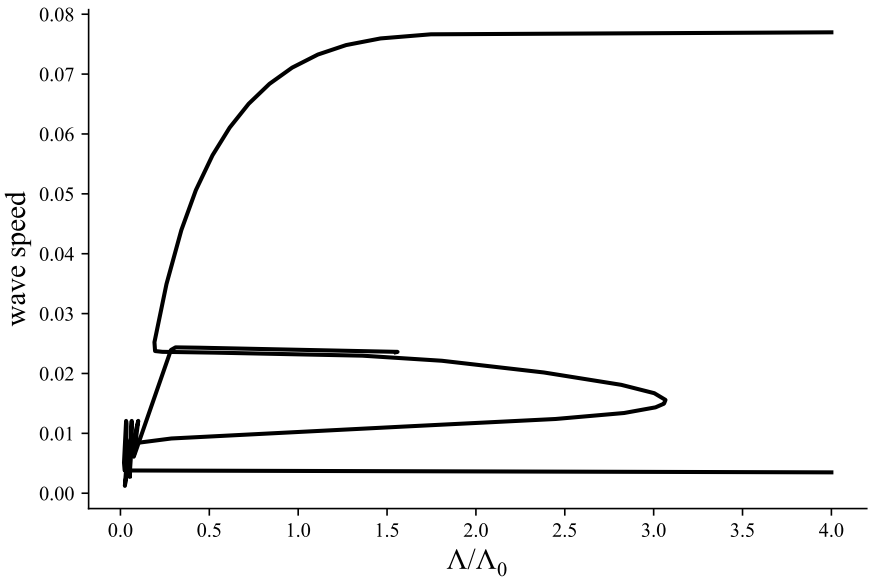


Figure 7: Multi-brach dispersion curve (without stability) in the reduced ten-Tusscher model.

3.8 The `visualize` Command

By default, TWIST supports interactive plotting capabilities. The `visualize` command will open a window where a user can select and view individual solutions along the dispersion curve. The window will show a dispersion curve and a solution profile that corresponds to a colored point on the dispersion curve. If the spectra have been computed, the eigenvalues will be shown as well.

If **only the dispersion curve is wanted**, pass in the `--individual` flag to the `visualize` command to split each subplot into its own figure and pass in the `--fixed-solutions` argument without specifying any solutions. For the full list of capabilities of the `visualize` command, see its help menu with `twist visualize --help`.

The following keyboard shortcuts are enabled by default for the plots generated by the visualize command:

Keyboard Shortcut	Function
Clicking on curve	Show solution to where the dispersion curve was clicked
Up arrow/n	Go to next solution
Down arrow/b	Go to previous solution
i	Print information about currently selected solution
Tab	Save currently selected solution to the plot
shift then <digits> then shift	Set the shown solution profile to the `<digits>`-th state (zero indexed)
Escape	Reset the plot back to starting point

Continued on next page

Keyboard Shortcut	Function
q	Close window
s	Save plot (supported file formats are png, ps, eps, svg and pdf)
p	Toggle pan mode
o	Toggle zoom-to-rect mode
g	Toggle grid lines
l	Toggle log-scale for y-axis
k	Toggle log-scale for x-axis
f	Toggle fullscreen
hold x	Constrain zoom-to-rect to x-axis
hold y	Constrain zoom-to-rect to y-axis

Note: Users can also **fully customize** the style of the plots by providing a "run-commands" file to the `visualize` command with the `--rc-file` argument. The plotting is done using matplotlib. For the full documentation of styling plots and setting run commands via `rc` files, see `https://matplotlib.org/stable/users/explain/customizing.html`

3.9 User Provided Initial Waves

Similarly to other programs, TWIST also allows for a user to provide their own initial guess for the solver using the `import` command. The syntax is

```
twist import <path to spec file> <path to data>
```

The supported data formats are `csv`, `tsv`, `dat` (space separated), `npy`, and `h5` and can be specified using the `-f/--file-type` flag.

For the text based formats (`csv`, `tsv`, `dat`), header information is not allowed and the first column is assumed to be the spatial points. Note that the difference between the last and first provided spatial points *must* be equal to the spatial period provided in the spec file. The latter columns in the provided data file must correspond to the values of each state (the keys in the `system` entry) in the spec file. The total number of columns must be equal to 1+the number of states (length of the `system` entry). An example user data file in CSV format for the Fitzhugh-Nagumo equations would look like

```
x0,v0,w0
x1,v1,w1
   ...
xN,vN,wN
```

When importing data using the `npy` format, the `npy` file should contain a matrix stored in the same manner as the text based formats above, i.e. first column corresponds to the grid points and the remaining columns correspond to the initial profiles. Both row major and column major ordering are supported. *Note:* numpy stores data using row major ordering by default.

When importing data using the HDF5 (`h5`) format, the following hierarchy must be used:

`/data/x`	grid points
`/data/y`	Initial profiles (must be stored in row major in same format as text files above)

The `import` command will automaticall derivative information generate the necessary spatial derivatives for the diffusive variables. However, a user can also provide spatial derivative information in the provided data file. The `--derivatives-included` flag needs to be passed to let TWIST know not to compute the derivatives and to assume the column immediately following each diffusive variable corresponds to its spatial derivative. An example of the expected format in the Fitzhugh-Nagumo equations where only v is diffusive would be

```
x0,v0,vprime0,w0
x1,v1,vprime1,w1
...
xN,vN,vprimeN,wN
```

The `import` command will also attempt to approximate the wave speed using the following equation,

$$c = \frac{\langle D\mathbf{u}_{xx} + G(\mathbf{u}, \alpha), \mathbf{u}_x \rangle}{\langle \mathbf{u}_x, \mathbf{u}_x \rangle} \tag{2}$$

Here $\mathbf{u}$, G, and α are the same as in equation 3. Note that this approximation can be very inaccurate so it is best to provide an approximation to the wave speed using the `-c/--wave-speed` command line argument.

3.10 Common Errors and Mistakes

The the goal of the spec files that TWIST uses to store the details of a model aim to be as simple and the least error prone, but that does not mean they are perfect. The following errors are common errors/typos with instructions on how to fix them.

Error	Cause	Fix
"undeclared identifier" or "not declared"	Most likely due to a typo in the spec file, forgetting to define constants, parameters, auxillary functions, ..., or defining an auxillary function that depends on another auxillary function that is defined after.	Ensure that each parameter and constant being used in `system` and `aux` have been defined in `constants` or `parameters`. Everything is *case senstive*. Make sure the cases match for *everything*. Verify that whatever auxillary functions are being used inside of other auxillary functions are defined prior.
"Derivative(...) Not supported ... "	Auxillary functions being defined in terms of other auxillary functions that haven't been defined yet.	Verify that whatever auxillary functions are being used inside of other auxillary functions are defined prior.
Action potential doesn't completely depolarize when initializing a wave	Likely that not enough was applied to the system or the system is not excitable.	Increase the amplitude of the applied stimulus using the `--stim-amp` command line argument when using `initialize` command and ensure that the parameter set being can be used to generate traveling waves. Also consider importing the traveling wave profile data using the `import` command.
Wave keeps collapsing during initialization	Traveling waves aren't supported for the given parameters, starting profile for the PDE simulation is too small/big, adaptive mesh algorithm is misbehaving	Make sure the parameters being used support traveling waves, decrease/increase `--time-ode` to make starting wave wider/skinnier, use a fixed mesh with command line argument `-m/--method` (use any method in the help menu that ends in "_n" to use a fixed mesh).
Continuation command outputs "failed" repeatedly	Dispersion curve branch has likely reached an existence boundary that does not have a turning point, or too large of a step size is being taken.	Try again with a smaller `ds` by lowering `--dsmax` when using the `continuation` command. If it persists, likely at an existence boundary. Stop and save current results using a keyboard interrupt (ctrl+c).

Continued on next page

Error	Cause	Fix
Solver fails to reach error tolerances	System is too ill-conditioned or magnitudes of traveling wave is too large.	Increase the error tolerance for the solver using the `--solve-tol` command line argument or rebuild the model using the `transforms` option in the spec file to make the traveling wave solution profiles roughly between zero and one.

3.11 Continuation Data Storage Format

The data generated by the `continuation` command to describe dispersion curves is stored in a consistent format shown in Table 3. This data can be used by users to further analyze or plot the data using any tool that supports the HDF5 format.

Table 3: HDF5 file structure for data generated during continuation.

Group	Dataset	Description
Dispersion curve points (labeled /0, /1, ...)		
/k/	`h`	Size of subintervals used for this solution
	`nnodes`	Number of nodes used for this mesh
	`nullspacev`	Nullspace vector at this point on the dispersion curve
	`p`	Parameter values at this point
	`state_curr`	State vector containing wave profiles and free parameter values
	`state_next`	*Ignore*: used internally by TWIST
	`state_prev`	*Ignore*: used internally by TWIST
	`stype`	Type of solution stored
	`t`	"time" points for the mesh
	`ypold`	Derivative values of `state_curr`
	`spectrum`	Eigenvalues for this traveling wave solution
Model configuration information (/basic_ info)		
/basic_info/	`diffusion_coeffs`	Diffusion coefficients
	`diffusion_indices`	Indices of diffusive variables
	`name`	Name of model
	`ndiff`	Number of diffusive variables
	`node`	Number of states (after being put into first order system)
	`np`	Total number of parameters
	`nparam`	Number of free parameters
	`nstages`	Number of collocation points
	`pmask`	Indices of free parameters in `p` vectors
	`spatial_period`	Default spatial period
Useful bookkeeping information (/meta_ data)		
/meta_data/	`cmdline`	The command line input that generated this data
	`directory`	The directory that this data was generated from
	`file_type`	Type of data being stored in this file

Note that in Table 3, the spectrum data stored under `/k/spectrum` contains three datasets: `alphar`, `alphai`, and `beta`. The eigenvalues are given by $\lambda = \frac{1}{\beta}(\alpha_r + i\alpha_i)$. However, some care must be taken when dividing by the values in `beta` because they can be zero (or near zero).

Part II
Numerical Methods

4 What Problem TWIST Solves

4.1 Traveling Waves

TWIST was designed to compute traveling wave solutions in general reaction-diffusion systems that are given by

$$\begin{aligned} \mathbf{u}_t &= \underbrace{D\mathbf{u}_{xx}}_{\text{diffusion}} + \underbrace{G(\mathbf{u},\alpha)}_{\text{reaction}} \\ 0 &= b(\mathbf{u}(t,x),\alpha) \end{aligned} \tag{3}$$

Here, $\mathbf{u} : [0,\Lambda] \times [0,\infty) \to \mathbb{R}^n$ is the solution to (3), $D \in \mathbb{R}^{n\times n}$ is a nonnegative, diagonal matrix, $F : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^n$ is a set of equations that govern the local reaction dynamics, $\alpha \in \mathbb{R}^p$ is a vector containing any paramters in the system, and $b : L^2([0,L]) \times \mathbb{R}^p :\to \mathbb{R}^n$ describes the boundary conditions on the system. Currently, TWIST only supports periodic boundary conditions given by

$$\begin{aligned} 0 &= \mathbf{u}(t,0) - \mathbf{u}(t,L) \\ &= b(\mathbf{u}(t,x),\alpha)\ . \end{aligned} \tag{4}$$

However, simple changes to the program will be made in the future to also support projection boundary conditions so that traveling pulses can be computed in addition to traveling waves. For a thorough explanation of projection boundary conditions, see [12].

Traveling waves (and pulses) are solutions to (3) that translate across space with a fixed profile at a constant velocity c. Because of this, $\mathbf{u}(t,x) = \mathbf{u}(x+ct)$, and the change of variable, $\xi = x + ct$, can be performed to move (3) into a moving coordinate system. The moving coordinate system (often called traveling wave coordinates) causes the wave to effectively be frozen in space. This change of variable also converts the system of PDEs, (3) into the following system of ODEs,

$$\begin{aligned} c\mathbf{u}_\xi &= D\mathbf{u}_{\xi\xi} + G(\mathbf{u},\alpha) \\ 0 &= b(\mathbf{u}(\xi),\alpha) \end{aligned} \tag{5}$$

In general, ODEs are easier to work with in both numerical and analytical contexts, and solving the two point boundary value problem (BVP), (5) will result in the solution, $\mathbf{u}(\xi)$.

4.2 Spectra of Traveling Waves

The stability of a traveling wave solution is determined in a manner analogous to the stability of a steady state in a finite-dimensional dynamical system. Suppose that a traveling wave solution, $\tilde{\mathbf{u}}$, has been computed, so that $\tilde{\mathbf{u}}$ is a solution to (5). Linearizing equation (5) about $\tilde{\mathbf{u}}$ leads to the eigenvalue problem,

$$\begin{aligned} \lambda\mathbf{q} &= D\mathbf{q}_{\xi\xi} - c\mathbf{q}_\xi + F_u(\tilde{\mathbf{u}},\alpha)\mathbf{q} \\ \mathbf{q}(0) &= \mathbf{q}(\Lambda) \end{aligned} \tag{6}$$

Here, $\mathbf{q}$ is an eigenfunction with eigenvalue λ, F_u corresponds to the Jacobian matrix of the reaction dynamics, F, linearized about $\mathbf{u}$. The solution, $\tilde{\mathbf{u}}$, is stable if and only if the spectrum is nonpositive. That is, $\Re(\lambda) \le 0$ for every $\lambda \in \mathbb{C}$. The eigenvalues and eigenfunctions associated with (6) characterize the instabilities that lead to qualitative changes in wave propagation.

5 Numerical Approaches Used by TWIST

The way TWIST approaches solving for traveling wave solutions in (3) is to first split the solution vector into its diffusive and nondiffusive states. Assuming periodic boundary conditions, the system can be rewritten as

$$\begin{aligned} \mathbf{v}_t &= D\mathbf{v}_{xx} + f_1(\mathbf{v}, \mathbf{w}, \alpha) \\ \mathbf{w}_t &= f_2(\mathbf{v}, \mathbf{w}, \alpha) \end{aligned} \tag{7}$$

The change of variables to traveling wave coordinates applied to (7) yields the system:

$$\begin{aligned} c\mathbf{v}_\xi &= D\mathbf{v}_{\xi\xi} + f_1(\mathbf{v}, \mathbf{w}, \alpha) \\ c\mathbf{w}_\xi &= f_2(\mathbf{v}, \mathbf{w}, \alpha). \end{aligned} \tag{8}$$

From here the second order system is converted into a first order system by defining $\mathbf{v}_\xi(\xi) = u(\xi)$.

$$\begin{aligned} \mathbf{v}_\xi &= \mathbf{r} \\ \mathbf{r}_\xi &= D^{-1}(c\mathbf{r} - f_1(\mathbf{v}, \mathbf{w}, \alpha)) \\ \mathbf{w}_\xi &= \frac{1}{c} f_2(\mathbf{v}, \mathbf{w}, \alpha) \end{aligned} \tag{9}$$

A solution to the first order system, (9), can now be approximated using a wide variety of methods. However, in the proceeding subsection, we explain how a hybrid method of multiple shooting and orthogonal collocation is used to approximate a solution to (9).

5.1 Orthogonal Collocation

Because, these equations in (18) are nonlinear, and analytical solutions rarely exist, the equations must be solved using an iterative root-finding method such as Newton's method. Therefore, the equations in (18) can be rearranged to define the following residuals.

Suppose that a solution to $y_\xi = F(y(\xi), \alpha)$ is desired for $\xi \in [0, L]$. Here, y and F would correspond to the vector of states and their respective differential equations in equation (9). That is,

$$\begin{aligned} y &= (\mathbf{v}\ \mathbf{r}\ \mathbf{w})^T \\ F(y, \alpha) &= \begin{pmatrix} \mathbf{r} \\ D^{-1}(c\mathbf{r} - f_1(\mathbf{v}, \mathbf{w}, \alpha)) \\ \frac{1}{c} f_2(\mathbf{v}, \mathbf{w}, \alpha) \end{pmatrix}. \end{aligned}$$

The system of differential equations in (9) is nonlinear there analytical solutions do not exist in general. Therefore, numerical methods such as orthogonal collocation [3] are required.

Orthogonal collocation involves approximating $y(\xi)$ via polynomials and requires that y_ξ be satisfied exactly at the roots of a chosen Legendre polynomial [3]. Orthogonal collocation is a highly accurate and robust [4] method for approximating solutions to systems differential equations such as (9).

5.1.1 Shooting Methods

Another method for solving boundary value problems (BVPs) such such as (9) is via shooting methods [2]. Shooting methods treat boundary value problems (BVPs) as initial value problems (IVPs) and systematically updates the initial starting point for the time integation method until a solution that satifies the boundary conditions is found. Using a shooting method can be desirable because shooting methods are closely related to floquet maps that can reveal spectral properties about a system [12]. While shootings methods are simple, they have the drawback that integration methods are being applied to a potentially unstable system (the linearization of F, F_y, has eigenvalues with positive real part). Because of the coordinate transformation into traveling wave coordinates and space is being treated as time, (9) will be unstable. Therefore, shooting methods are not appropriate for more than one time (space in this case) step becuase numerical errors exponentially accumulate as more steps are performed. To avoid the error accumulations, the approximate solution is formulated as a single time step for each subinterval.

5.1.2 Orthogonal Collocation as a Runge-Kutta Based Shooting Method

Many initial value methods rely on the fact that,

$$y(\xi) - y(0) = \int_0^{\xi} F(y(\xi), \alpha) d\xi \tag{10}$$

and derive approximations to y by applying quadrature methods to the integral in (10). More accurate solutions can be evaluated by splitting the domain $[0, \Lambda]$ into subintervals

$$0 = \xi_0 < \xi_1 < \cdots < \xi_{N_{\text{grid}}-2} < \xi_{N_{\text{grid}}-1} = \Lambda$$

using N_{grid} points. The single integral in (10) can be split into a sub of integrals on each subinterval,

$$\int_0^{\Lambda} F(y(\xi), \alpha) d\xi = \sum_{k=0}^{N_{\text{grid}}-2} \int_{\xi_k}^{\xi_{k+1}} F(y(\xi), \alpha) d\xi \ . \tag{11}$$

and applying the same quadrature method on each subinterval.

A highly accurate quadrature method is Gaussian quadrature [9]. Gaussian quadrature is based on first approximating $F(y(\xi), \alpha)$ via Lagrange interpolation at the roots of a Legendre polynomial of a chosen degree. For the degree s Legendre polynomial, there are s roots (collocation points) the following approximation is used

$$\begin{aligned} F(y(\xi), \alpha) &\approx \sum_{i=1}^{s} F(y(\xi_{k,i}), \alpha) \ell_{k,i}(\xi), \quad \xi \in [\xi_k, \xi_{k+1}] \\ \ell_{k,i}(\xi) &= \prod_{\substack{j=1 \\ j \neq i}}^{s} \left(\frac{\xi - \xi_{k,j}}{\xi_{k,i} - \xi_{k,j}} \right) \end{aligned} \tag{12}$$

Here $\xi_{k,i} = \frac{1}{2}((\xi_{k+1} - \xi_k)c_i + (\xi_{k+1} + \xi_k))$ where c_i is the i-th root of the Legendre polynomial of order s. Substituting (12) into (11) yields for each integral

$$\begin{aligned} \int_{\xi_k}^{\xi_{k+1}} F(y(\xi), \alpha) d\xi &\approx \int_{\xi_k}^{\xi_{k+1}} \sum_{i=1}^{s} F(y(\xi_{k,i}), \alpha) \ell_{k,i}(\xi) \\ &= \sum_{i=1}^{s} F(y(\xi_{k,i}), \alpha) \int_{\xi_k}^{\xi_{k+1}} \ell_{k,i}(\xi) \\ &= \sum_{i=1}^{s} F(y(\xi_{k,i}), \alpha) \underbrace{(\xi_{k+1} - \xi_k)}_{h_k} \underbrace{\int_0^1 \tilde{\ell}_i(\xi)}_{b_i} \\ &= h_k \sum_{i=1}^{s} b_i F(y(\xi_{k,i}), \alpha) \end{aligned} \tag{13}$$

and

$$\tilde{\ell}_i(\xi) = \prod_{\substack{j=0 \\ j \neq i}}^{s} \left(\frac{\xi - c_j}{c_i - c_j} \right) \tag{14}$$

In equation (13), $y(\xi_{k,i})$ are still undefined. However, they can also be approximated using quadrature rules

$$\begin{aligned} y(\xi_{k,i}) - y(\xi_k) &= \int_{\xi_k}^{\xi_{k,i}} F(y(\xi), \alpha) d\xi \\ &\approx \sum_{j=1}^{s} F(y(\xi_{k,i}), \alpha) \int_{\xi_k}^{\xi_{k,i}} \ell_{k,j}(\xi) \\ &\approx h_k \sum_{j=1}^{s} F(y(\xi_{k,i}), \alpha) \underbrace{\int_0^{c_i} \tilde{\ell}_j(\xi)}_{a_{i,j}} \\ &= h_k \sum_{j=1}^{s} a_{i,j} F(y(\xi_{k,i}), \alpha) \end{aligned} \tag{15}$$

The quadrature approximations in equations (13) and (15) lead to the Runge-Kutta method

$$\begin{aligned} y_{k+1} &= y_k + h_k \sum_{i=1}^{s} b_i F(y_{k,i}, \alpha) \\ y_{k,i} &= y_k + h_k \sum_{j=1}^{s} a_{i,j} F(y_{k,j}, \alpha) \quad i = 1, \ldots, s \end{aligned} \tag{16}$$

Here, y_k, $y_{k,i}$, and y_{k+1} are approximations to $y(\xi_k)$, $y(\xi_{k,i})$, and $y(\xi_{k+1})$ respectively. The Runge-Kutta method in (16) can be summarized by the following Butcher tableau

$$\begin{array}{c|ccc} c_1 & a_{1,1} & \ldots & a_{1,s} \\ \vdots & \vdots & & \vdots \\ c_s & a_{s,1} & \ldots & a_{s,s} \\ \hline & b_1 & \ldots & b_s \end{array} \tag{17}$$

This class of Runge-Kutta methods is equivalent to the method of orthogonal collocation and are typically referred to as Gauss-Legendre Runge-Kutta (GL) methods.

The reason that the Runge-Kutta representation is used instead of using the Lagrange polynomials directly is multifactorial:

1. The Runge-Kutta formulation is also equivalent to a multiple shooting method that takes a single step for each subinterval, this means the same discretization can be used during linear stability analysis to get a highly accurate approximation to the spectrum. A high accuracy spectrum implies reliable and accurate bifurcation analysis can also be performed if desired.

2. Computing the essential spectrum requires a trivial modification to the boundary condition ($y(0) = y(\Lambda)$ becomes $e^{i\gamma} y(0) = y(\Lambda)$, $\gamma \in \mathbb{R}$) and can be computed much more efficiently because most complex arithmemtic can be avoided

5.1.3 Forming the Nonlinear System of Equations

Recall that analytical solution are typically not available and that (9) is nonlinear. Therefore, to compute a numerical approximation to the nonlinear system of equations, a root-finding method such as Newton's method must be used.

Discretizing $y_\xi = F(y(\xi), \alpha)$ using GL methods leads to a large system of nonlinear equations. On each subinterval, there are two sets of equations: the collocation equations and the continuity (quadrature) equation. These correspond to the upper block and last row respectively in the Butcher tableau (17) and

for the k-th subinterval are given by

$$\begin{aligned}
\text{Collocation Equations:} \quad & y_{k,i} = y_k + h_k \sum_{j=1}^{s} a_{i,j} F(y_{k,j}, \alpha) \quad i = 1, \ldots, s \\
\text{Continuity Equation:} \quad & y_{k+1} = y_k + h_k \sum_{i=1}^{s} b_i F(y_{k,i}, \alpha) \\
& k = 0, \ldots, N_{\text{grid}} - 2 \\
\text{Boundary Conditions:} \quad & 0 = y(0) - y(\Lambda)
\end{aligned} \tag{18}$$

The continuity and collocation equations in (18) are rearranged to define the following residuals that can be used in a root-finding algorithm.

$$\begin{aligned}
R_{k,0} &\equiv y_k - y_{k+1} + h_k \sum_{i=1}^{s} b_i F(y_{k,i}, \alpha) \\
R_{k,i} &\equiv y_k - y_{k,i} + h_k \sum_{j=1}^{s} a_{i,j} F(y_{k,j}, \alpha) \quad i = 1, \ldots, s
\end{aligned} \tag{19}$$

5.1.4 Forming the Linear System for a Root-Finding Method

Iterative methods such as Newton's method, rely on linearly approximating the nonlinear residuals using via the Jacobian matrix. The residuals defined in equation (19) for the k-th subinterval only depend on the two nodes at the left and right endpoints (y_k and y_{k+1}), and the intermediate "stages" ($y_{k,i}$'s). Therefore, the Jacobian matrix will be sparse and can be defined in terms of its "blocks."

$$\begin{aligned}
&\frac{\partial R_{k,0}}{\partial y_k} = I \qquad \frac{\partial R_{k,0}}{\partial y_{k,j}} = h_k b_j F_y(y_{k,j}, \alpha) \qquad \frac{\partial R_{k,0}}{\partial y_{k+1}} = -I \qquad \frac{\partial R_{k,0}}{\partial \alpha} = h_k \sum_{i=1}^{s} b_j F_\alpha(y_{k,j}, \alpha) \\
&\frac{\partial R_{k,i}}{\partial y_k} = I \qquad \frac{\partial R_{k,i}}{\partial y_{k,j}} = h_k a_{i,j} F_y(y_{k,j}, \alpha) - \delta_{i,j} I \qquad \frac{\partial R_{k,i}}{\partial y_{k+1}} = 0 \qquad \frac{\partial R_{k,i}}{\partial \alpha} = h_k \sum_{i=1}^{s} a_{i,j} F_\alpha(y_{k,j}, \alpha)
\end{aligned} \tag{20}$$

Figure 8 illustrates the sparity pattern generated by these blocks. The derivatives with respect to α are also needed becuase there are often unknown parameters that will also need to be solved for in the system such as the wave speed, c, in equation (9).

5.1.5 Phase Condition

An important consideration for traveling wave solutions is that they are not unique due to the translational invariance of the system and boundary conditions. If $y(\xi)$ is a traveling wave solution of (9), then for any $\sigma \in \mathbb{R}$, $y(\xi + \sigma)$ is also a solution of (9). Therefore, a method to lock the phase is needed. A popular and robust [7] method is an integral phase condition. To lock the phase, a "reference" function, $\hat{y}(\xi)$, is introduced with a similar profile (not required but it helps with intuition) to the desired traveling wave solution, $y(\xi)$. To ensure that the phase of $y(\xi)$ is fixed, the distance between $y(\xi)$ and $\hat{y}(\xi + \sigma)$ is minimized. That is,

$$D(\sigma) = \frac{1}{2} \int_0^\Lambda \langle y(\xi) - \hat{y}(\xi + \sigma), y(\xi) - \hat{y}(\xi + \sigma) \rangle d\xi \tag{21}$$

is minimized. $D(\sigma)$ will be minimized at some value $\hat{\sigma}$, and $D_\sigma(\hat{\sigma}) = 0$. Computing $D_\sigma(\hat{\sigma}) = 0$ yields,

$$\begin{aligned}
0 &= \frac{1}{2} \int_0^\Lambda \frac{d}{d\sigma} \langle y(\xi) - \hat{y}(\xi + \hat{\sigma}), y(\xi) - \hat{y}(\xi + \hat{\sigma}) \rangle d\xi \\
&= \frac{1}{2} \int_0^\Lambda -2 \langle \hat{y}_\xi(\xi + \hat{\sigma}), y(\xi) - \hat{y}(\xi + \hat{\sigma}) \rangle d\xi \\
&= \int_0^\Lambda \langle \hat{v}_\xi(\xi), y(\xi) - \hat{v}(\xi) \rangle d\xi \quad \text{(phase condition)}
\end{aligned} \tag{22}$$

This phase condition is added to the system of nonlinear equations to make the solution locally "unique" which is required for root finding methods. Moreover, because the wave speed, c, is an unknown unknown parameter that needs to be solved for, the system of equations is no longer underdetermined with the addition of the phase condition. Lastly, because the traveling wave solution, $u(\xi)$ is being approximated via Gaussian quadrature, (22) can be well approximated using the same scheme:

$$\begin{aligned}\int_0^{\Lambda} \langle \hat{y}_\xi(\xi), y(\xi) - \hat{y}(\xi)\rangle d\xi &= \sum_{k=0}^{N_{\text{grid}}-2} \int_{\xi_k}^{\xi_{k+1}} \langle \hat{y}_\xi(\xi), y(\xi) - \hat{y}(\xi)\rangle d\xi \\ &\approx \sum_{k=0}^{N_{\text{grid}}-2} h_k \sum_{i=1}^{s} b_i \langle (\hat{y}_\xi)_{k,i}, y_{k,i} - \hat{y}_{k,i}\rangle \\ &= \sum_{k=0}^{N_{\text{grid}}-2} h_k \sum_{i=1}^{s} b_i \sum_{j=1}^{N_{\text{sys}}} (\hat{y}_\xi)_{k,i,j} \ (y_{k,i} - \hat{y}_{k,i,j})\end{aligned} \tag{23}$$

Here N_{sys} represents the total number of variables in the system of differential equations (9). Equation (23) is also a linear combination of already computed values, therefore, its partial derivatives for the Jacobian matrix are also trivual.

5.2 Globalized Newton's Method

The residuals defined in equations (19) are nonlinear and analytical solutions are not available. Therefore, a root-finding algorithm is necessary to numerically approximate a solution.

A well established method for solving nonlinear equations is Newton's method.

$$z_{n+1} = z_n - (J(z_n))^{-1} R(z_n) \tag{24}$$

Here z_n is a vector containing the approximation of each y_k, $y_{k,i}$, and the wave speed c after the n-th iteration of Newton's method. $R(z)$ is a vector function containing the residuals $R_{k,i}$ and phase condition. $J(z)$ is the linearization of $R(z)$.

The basins of attraction for Newton's method can become increasingly smaller as the as the derivitives become less Lipschitz continuous (more unbounded). This implies that as the nonlinear equations become "more" nonlinear, Newton's method will require a more accurate starting guess. A common property of high-dimensional, biophysically detailed models of cariac tissue is their larger differences in time-scales. This means that the individual differential equations will often have very different magnitudes which can result in large Lipschitz constants for the Jacobian matrices. Therefore, unless a sufficiently accurate initial guess to a traveling wave and its wave speed is provided, classical Newton's method will fail to converge to a solution.

To mitigate this issue, and greatly lessen the strictness on providing an accurate initial guess, TWIST utilizes a global Newton's method that incorporates a "damping" factor, $0 < \lambda_n \leq 1$. The modifued method is given by

$$z_{n+1} = z_n - \lambda_n (J(z_n))^{-1} R(z_n) \tag{25}$$

Here, λ_n expands the size of the basin of attraction by not allowing the next guess in the iteration to stray too far from the previous guess. The damping factor, λ_n, is chosen using a combination of error oriented and residual based descents [6]. For the residual based descent, the simple descent test

$$\|R(z_{n+1})\| \leq \|R(z_n)\| \tag{26}$$

is used. This approach is the simplest and does make the method "global." However, using only this condition, can quickly lead to stalling as $\lambda_n \to 0$. This is often the case for quasi-Newton methods. In the context of computing traveling wave solutions, the Jacobian matrices can be very ill-conditioned leading to numerical rounding errors that mimic errors in quasi-Newton methods. Thus, (26) is paired with the affine invariant natural monotonicity test

$$\|(J(z_n))^{-1} R(z_{n+1})\| \leq \left(1 - \frac{1}{2}\lambda_n\right) \|(J(z_n))^{-1} R(z_n)\| \tag{27}$$

The combination of conditions (26) and (27) not only prevents overshoot that is common in Newton's method, but can also help the residuals reach norms that are close to machine precision. Having small norms is important, because small pertubations to the traveling wave solution can result in very large changes in the spectrum. Algorithm 1 outlines the pseudocode for the global Newton's method that is implemented in TWIST.

Algorithm 1: Gobal Newton method for computing traveling wave solutions

Input: z_0, $R(z)$, ε
Output: Root to nonlinear equations z

$z_n = z_0$
$R_n = R(z_n)$
if $\|R_n\| < \varepsilon$ **then**
  return
end if
$\lambda = 1$
for $n_{\text{iter}} = 0, \ldots$ **do**
  $J = \frac{dR}{dz}(z_n)$
  $\Delta = J^{-1} R_n$
  $\lambda = \min(2\lambda, 1)$
  while $\lambda_{\min} < \lambda$ **do**
    $z_{n+1} = z_n - \lambda\Delta$
    $R_{n+1} = R(z_{n+1})$
    $\bar{\Delta} = J^{-1} R(z_{n+1})$
    if $\|\bar{\Delta}\| \leq (1 - \frac{1}{2}\lambda)\|\Delta\|$ and $\|R_{n+1}\| < \|R_n\|$ **then**
      break
    end if
    $\lambda = \frac{1}{2}\lambda$
  end while
  $z_n = z_{n+1}$
  $R_n = R_{n+1}$
  if $\|R_n\| < \varepsilon$ or $\|\Delta\| < \varepsilon$ **then**
    break
  end if
end for

5.3 Specialized Linear Solver

When (9) is discretized using orthogonal collocation, the result is a system of many nonlinear equations that has to be solved using an iterative such as the global Newton method in Algorithm 1. Part of any variant Newton's method is the solution of a linear system of equations. The standard approach to solving a linear system $Ax = b$ is with dense linear algebra algorithms such as the ones found in LAPACK [1]. For an $n \times n$ matrix, the computational time is on the order of $O(n^3)$. However, when the matrix, A, is sparse, the computation time can be reduced through the use of specialized solvers for the system, $Ax = b$. The Jacobian matrix that arises from linearizing (19) has a sparsity structure that is shown in Figure 8 and its almost block diagonal structure can be exploited to greatly reduce the computational time. The size of the Jacobian is roughly $n = N_{\text{sub}} N_{\text{sys}} N_{\text{col}}$ where N_{col} is the number of collocation points or the degree of the Legendre polynomial being used and $N_{\text{sub}} = N_{\text{grid}} - 1$.

The Jacobian matrix in Figure 8 can be factored using a recursive process. Each step partitions the matrix via the blocks along the diagonal and the smaller blocks that share the some rows and columns in the last few columns and rows. Figure 9a shows each of these partitions being color coded for clarity. The partitions are then stored in memory chunks, illustated in Figure 10, that can be factored using dense linear algebra routines. TWIST uses LU factorizations for each partition. Each of the partitions are stored

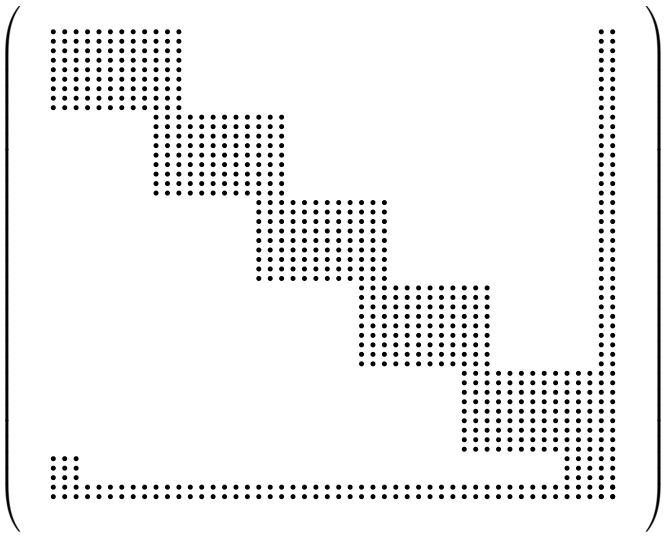


Figure 8: Example Jacobian matrix structure that needs to be factored during continuation. Here there are six grid points, two collocation points, and three state variables.

independently therefore each partition can be factored in parallel. After each partition has been factored, the columns corresponding to the y_k's have been decoupled from the interior collocation points, $y_{k,i}$'s, which results in a subsystem, highlighted in red in Figure 9b, that can be factored using the same method. This process is repeated recursively until only two nodes (and the unknown parameters) remain. The recursive nature of this algorithm implies that each layer in the recursion can be stored as a layer in a tree structure. The tree structure for the matrix in Figure 8 is shown in Figure 11. At the bottom of the tree in Figure 11, the final block will be small (and dense) enough such that if can be factored using existing high peformance dense matrix factorization algorithms such as those found in OpenBLAS [18]. After the final block has been factored, the factorized matrix can be used (and reused) to efficiently solve for the needed update vectors in Algorithm 1 by performing the necessary backsubstitutions by going from the bottom of the tree to the top.

The most expensive part of this process is the initial factorization of the first set of partition blocks. Each partition will take $O(N_{\text{sys}}^3 N_{\text{col}}^3)$ time to factor and there are N_{sub} partitions, therefore the total time will be $O(N_{\text{sub}} N_{\text{sys}}^3 N_{\text{col}}^3)$.

This strategy of recursively factoring the matrix and the resulting subsystems can lead to a large accumulation of rounding errors. Therefore, a pivoting strategy is needed to reduce the growth factors [10]. To maintain efficiency and stability, the rook pivoting strategy is used for the initial set of partitions because these blocks will be the largest and rook pivoting is nearly as cheap as partial pivoting but provides similar stability to complete pivoting in practice [8, 15]. Then to ensure stability stability for the rest of the layers in the recursion, complete pivoting is used on much smaller sized partition blocks.

To compare the accuracy and speed of the factorization strategy above, benchmarks were performed for three different solve methods:

1. Direct solver: The non-symmetric sparse solver provided in UMFPack [5]
2. Hybrid solver: Use condensation on the initial large partitions then use direct solver on the first subsystem
3. Tree solver: method described above

The tree based solver was the fastest in every case and the computation runtimes grew linearly with the number of subintervals. The other solvers appear to grow linearly at for a smaller number of subintervals, but then become superlinear, then quadratic in Figure 12.

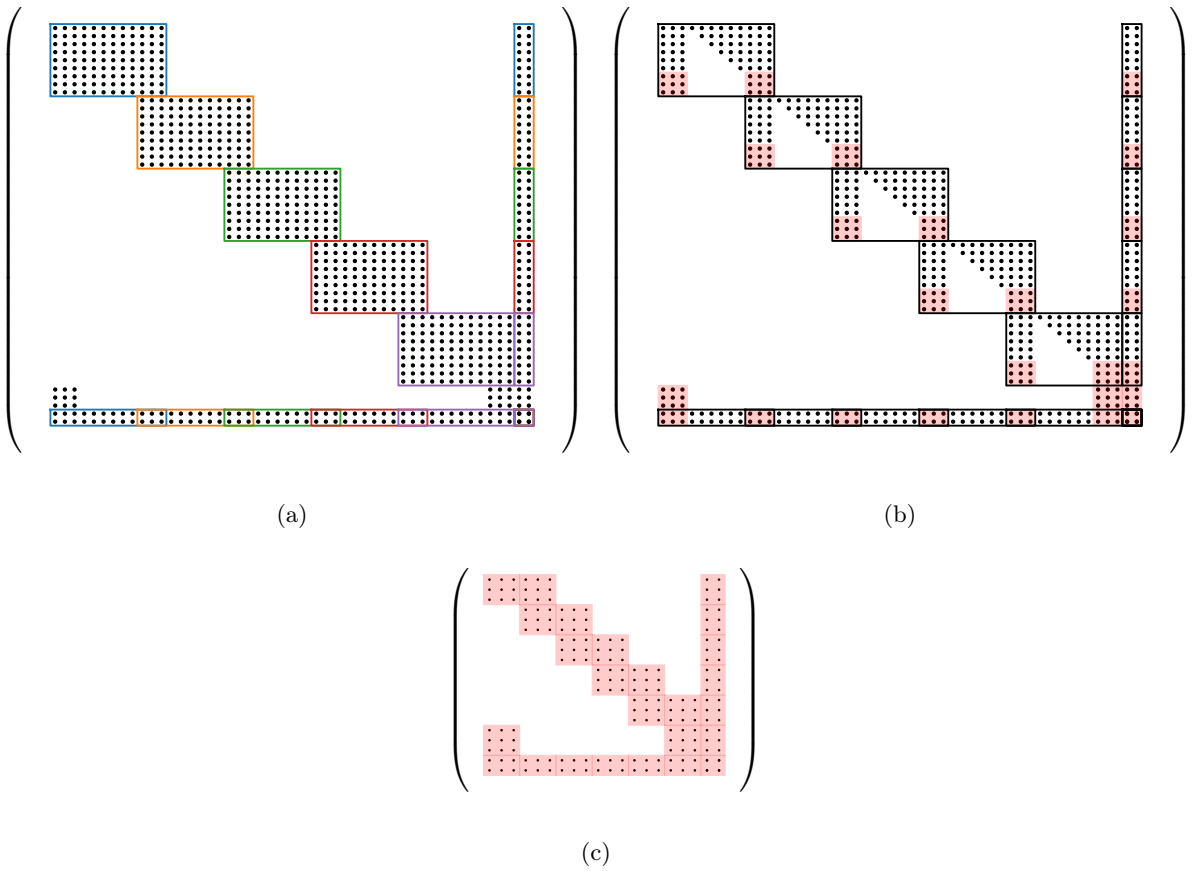


Figure 9: (a) Initial partitioning of the matrix from Figure 8. Each color represents one of the five partitions. (b) After the initial factorization of each partition is done, the subsystem shaded in red can be recursively factored. (c) The subsystem matrix from(b) that can be recursively condensed.

5.4 Determining Stability of Traveling Waves

In this section we use a Fitzhugh-Nagumo like system (one diffusive variable and one non-diffusive variable) to illustrate how TWIST computes the stability of a traveling wave.

The appoach that TWIST uses to determine the stability of a traveling wave solution is by first linearizing equation (8) about a traveling wave solution, $(\tilde{v}, \tilde{w})$ which results in the eigenvalue problem,

$$\begin{aligned} \lambda v &= Dv_{\xi\xi} - cv_{\xi} + f_v(\tilde{v}, \tilde{w}, \alpha)v + f_w(\tilde{v}, \tilde{w}, \alpha)w \\ \lambda w &= -cw_{\xi} + g_v(\tilde{v}, \tilde{w}, \alpha)v + g_w(\tilde{v}, \tilde{w}, \alpha)w \end{aligned} \tag{28}$$

The standard approach used by various software packages is to estimate the spectrum by discretizing the derivative operators in equation (28) using finite difference methods (sometimes Fourier differentiation matrices). However, this approach, while simple, does not lead to accurate approximations of the spectrum due to switching discretizations and rounding errors that this approach often experiences. Moreover, many grid point must be used in order to accurately approximate the steep gradients that can occur, leading to large computational times and sometimes failure. These limitations can be addressed by discretizing the eigenvalue problem in equation (28) with the same GL, discretization that was used for the nonlinear system, (9).

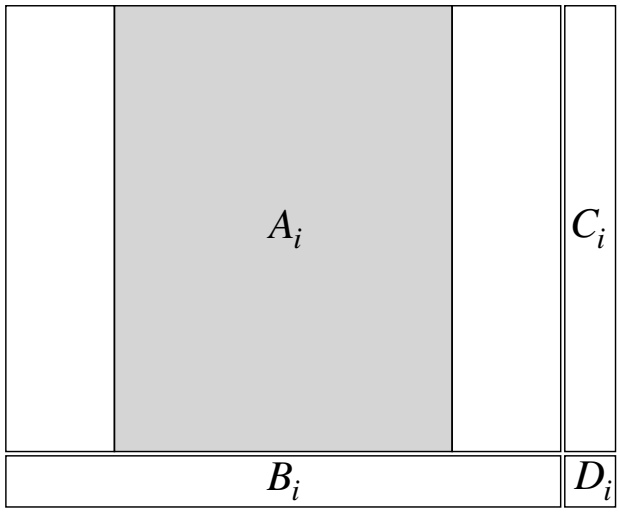


Figure 10: One partition from Figure 9a. The gray region is the pivoting window where rook pivoting is employed.

We rearrange (28) into the first order system

$$\lambda \underbrace{\begin{pmatrix} 0 & 0 & 0 \\ D^{-1} & 0 & 0 \\ 0 & 0 & -1/c \end{pmatrix}}_{B} \underbrace{\begin{pmatrix} v \\ r \\ w \end{pmatrix}}_{z} = \underbrace{\begin{pmatrix} v_\xi \\ r_\xi \\ w_\xi \end{pmatrix} - \begin{pmatrix} 0 & 1 & 0 \\ -D^{-1} f_v(\tilde{v}, \tilde{w}) & D^{-1} c & -D^{-1} f_w(\tilde{v}, \tilde{w}) \\ \frac{1}{c} g_v(\tilde{v}, \tilde{w}) & 0 & \frac{1}{c} g_w(\tilde{v}, \tilde{w}) \end{pmatrix}}_{A} \underbrace{\begin{pmatrix} v \\ r \\ w \end{pmatrix}}_{z} \tag{29}$$

which results in linear system of equations in the form of a generalized eigenvalue problem, $Az = \lambda Bz$. Note that the right hand side of equation (29) is equation (9) linearized about the same traveling wave solution. Therefore, the Jacobian matrix from Newton's method in the previous section can be used for the matrix, A, in this generalized eigenvalue problem. Furthermore, the matrix, B, can be computed by replacing $F(y, \alpha)$ in equation (9) with the left hand side in equation (29). That is, let

$$F(y, \alpha) = \begin{pmatrix} 0 & 0 & 0 \\ D^{-1} & 0 & 0 \\ 0 & 0 & -1/c \end{pmatrix}$$

The source of errors in the approximation of the spectrum that cannot be avoided are the final residual in the nonlinear equations after Algorithm 1 has been performed and the truncation errors in the discretization method being used. This approach to approximating the spectrum of the traveling wave has two important benefits:

1. The same discretization is being used to approximate the spectrum, therefore, no additional discretization errors will appear due to switching discretizations (without resolving)

2. High order GL methods can be used to approximate the traveling wave solution with a small number of subintervals, which leads to matrix pencils, (A, B), that are small enough to be used in standard dense linear algebra algorithms.

These benefits enable the hihgly accurate stability and bifurcation analysis of high-dimensional, biophysically detailed models that was previously computationally infeasible.

5.5 New Generalized Eigenvalue Problem Algorithm

Computing the solution to a generalized eigenvalue problem is a computationally expensive task. Furthermore, existing algorithms for computing the decomposition suffer from a lack of scalability and having to

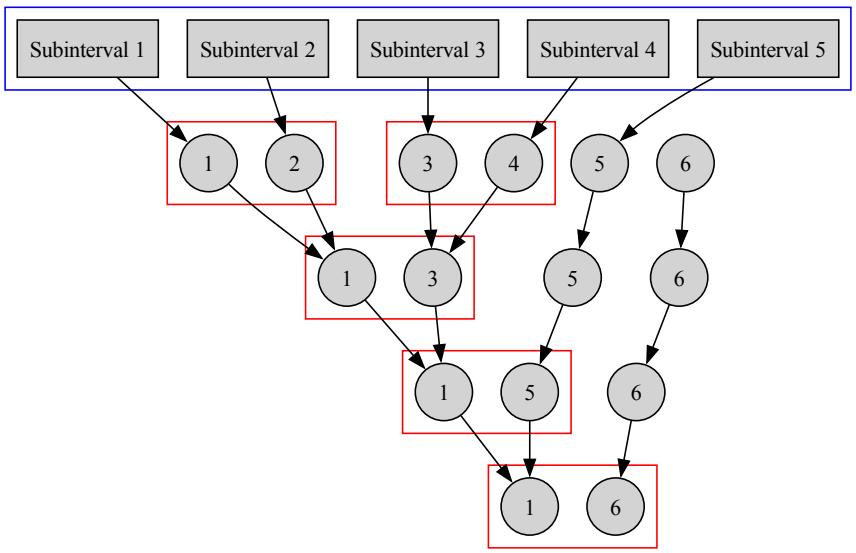


Figure 11: The tree structure used to perform the recursive factorization process for the subsystem in Figure 9b. Rectangles represent the initial large partitions for each subinterval during the condensation phase (Figure 9a). Circles represent the nodes. The red boxed regions in each layet of the tree represent the partitions for each subsystem during the recursive process. Nonboxed nodes correspond to trailing subsystem blocks and boundary conditions that get handled later in the tree.

handle the presense of eigenvalues at infinity. In this section a novel approach to the generalized eigenvalue problem is taken that greatly reduces the runtime and addresses each bottleneck of existing algorithms.

Currently, one of the most widely used method algorithm for computing the generalized eigenvalue problem,

$$Av = \lambda Bv \tag{30}$$

is the xGGEV3 algorithm in the LAPACK standard [1]. The xGGEV3 algorithm is comprised of the following high level steps:

1. $B = QR$: QR factorize B
2. $A = Q^*A$: Apply Q^* to A
3. Reduce (A, B) to Hessenberg-triangular (HT) form using the tiled HT algorithm from [11]
4. Perform QZ iterations until (A, B) is (quasi) upper triangular

Note that some steps pertaining the the eigenvectors have been omitted for brevity. In the steps outlined above, the reduction of (A, B) to HT form is the most computationally expensive. The current best performing algorithm [11] is a modification of the orginal HT algorithm by Moler and Stewart [14] that first accumulates a fixed number of Givens rotations into a matrix buffer that is then applied to the pencil using optimized matrix multiplication algorithms. This method is much more performant than the orginal algorithm. However, because the Givens rotations are being accumulated into a matrix of fixed, size the algorithm cannot scale as the size of the matrix pencil grows. Moreover, any potential singularities in B can be randomly distributed across the columns of B which can cause significant slowdowns during the final QZ step of the xGGEV3 algorithm. Therefore, an ideal algorithm for the generalized eigenvalue problem would be one that

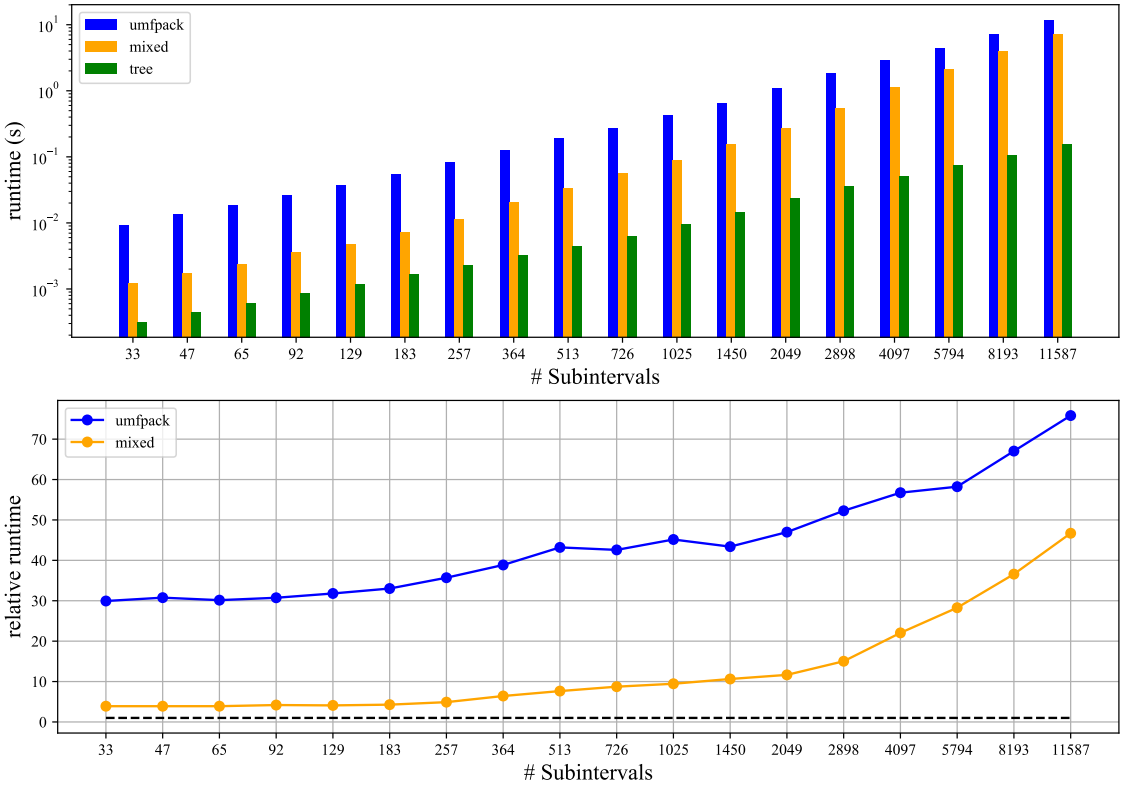


Figure 12: Benchmark results for three different algorithms for factoring the Jacobian matrices with varying number of subintervals. The recursive solver (tree) performs the best in every scenario. The hybrid (mixed) solver is the second most performant, however, it becomes nearly as slow as the UMFPack solver as the number of subintervals grows larger.

1. Has faster runtimes than existing algorithms
2. Is scalable as the size of the pencil grows
3. Does *not* get bottlenecked by the presense of eigenvalues at infinity

The majority of the runtime is spent on the HT reduction step (even with eigenvalues at infinity), therefore, improvements to the HT reduction will lead to the largest performance improvements relative to the remaining steps. Steel et al [17] proposed that the HT reduction can be performed using only matrix multiplication steps that act on the entire pencil. These steps are outlined in Algorithm 2. Algorithm 2 requires that B^{-1} be applied to A, however, B is often singular in many applications. Therefore, Algorithm 2 is not suited without considering a singular B.

Algorithm 2 Matrix multiplication based Hessenberg-triangular reduction [17]

Input: $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times n}$
Output: reduced (A, B)
$X = AB^{-1}$
Calculate unitary Q such that Q^*XQ is Hessenberg
$A = Q^*A$
$B = Q^*B$
Calculate unitary Z such that BZ is upper triangular
$A = AZ$
$B = BZ$

The rank revealing RQ (RRRQ) decomposition of B is used in [17] to expose the rank of B and move all of the singularities into the leftmost columns of the matrix. If the rank of the matrix is r, then the first $n-r$ columns of B will be zero.

$$PB = RQ \qquad R = \begin{pmatrix} 0 & R_{12} \\ 0 & R_{22} \end{pmatrix} \qquad |r_{i,i}| \leq |r_{i+1,i+1}| \tag{31}$$

TWIST exploits this structure for the generalized eigenvalue problem. When B is in this form, all of the eigenvalues at infinity can be immediately exposed and deflated. The deflation can be done by applying a QR factorization to the first $n-r$ columns of A (see Figure 13), then undoing the resulting fill-in in last r columns of B using an RQ decomposition (see Figure 14).

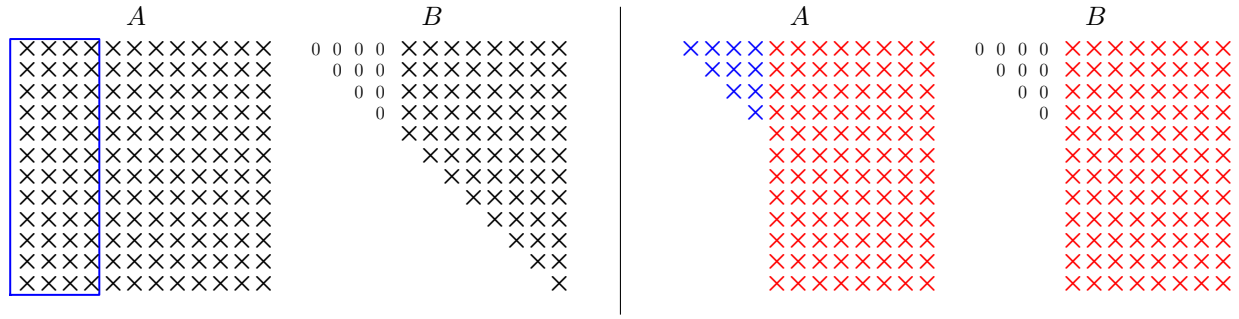


Figure 13: Deflation: Perform QR on first $n-r$ columns of A. Apply Q^T to remaining columns of A and last r columns of B. The first $n-r$ columns of A and B are now in upper triangular form and can be ignored by future reduction routines.

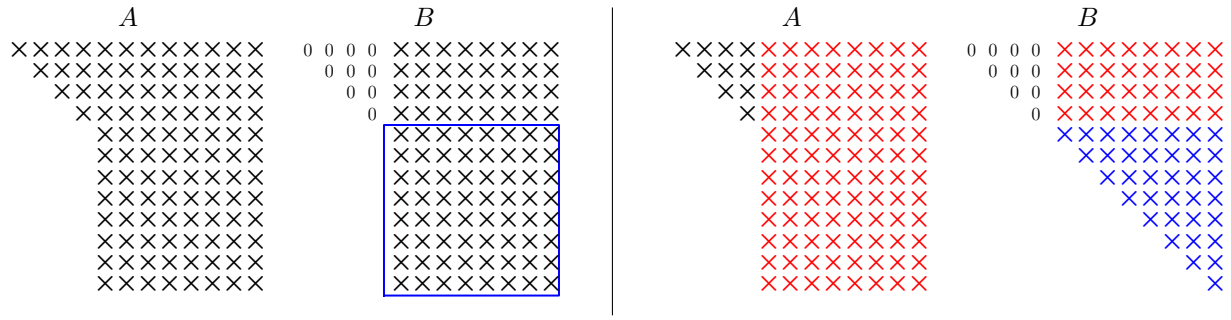
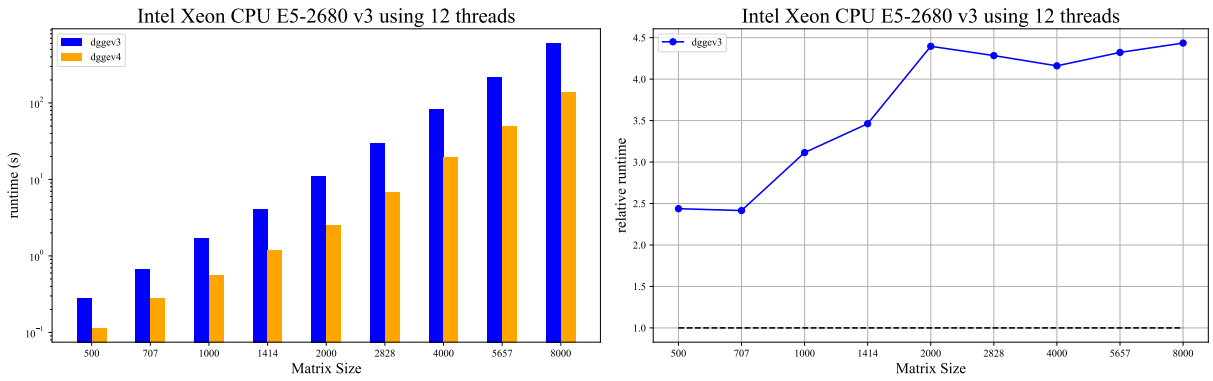


Figure 14: Deflation: Perform RQ on last r rows and columns of B. Apply Q^T to last r columns of A and last r columns and $n-r$ rows of B. This restores the upper-triangular structure of B without undoing the the upper-triangular structure of the leading columns of A.

After all of the singularities of B have been handled and all of the eigenvalues at infinity have been deflated, Algorithm 2 can be iteratively applied to the remaining submatrix as outlined in [17]. After the pencil has been reduced to HT form, the QZ algorithm can be applied to reduced the pencil to upper triangular. Algorithm 3 outlines the complete algorithm (including the optional computation of the eigenvectors).

Almost every step xGGEV4 uses algorithms that are based on matrix multiplication that act on the entire (deflated) system. Therefore, when xGGEV4 is paired with fast and scalable matrix multiplication kernels, it will have good performance and better scaling properties than xGGEV3. Moreover, because all of the eigenvalues at infinity are being handled before the QZ phase, the QZ iterations should not be bottlenecked.

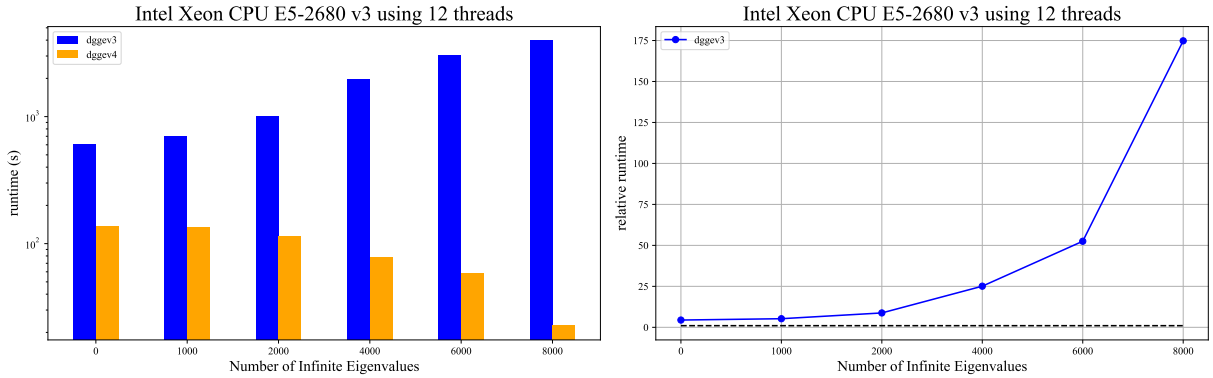
The xGGEV4 algorithm is significantly faster than the xGGEV3 algorithm experiencing large decreasing in runtimes. Figure 15b shows xGGEV4 is roughly 4.5 times faster than xGGEV3 on an Intel Xeon E5-2860 v3 CPU when using 12 threads. Similar speedups occur when limited to only 4 threads (about 3.5



(a) (b)

Figure 15: (a) Runtimes for the xGGEV3 and the xGGEV4 algorithms on full rank pencils. The xGGEV4 algorithm has lower runtimes in all cases. (b) The relative runtimes between the xGGEV3 and xGGEV4 algorithms. The relative runtime is defined as (time xGGEV3)/(time xGGEV4). xGGEV4 is about 4.5 times faster for sufficiently large matrices.

times faster). The bottleneck in during the QZ iteration step is removed by the deflation step. Figures 16a and 16b xGGEV4 experiences faster runtimes for a rank-deficient B than a full rank B, whereas, xGGEV3 gets slower. xGGEV4 has much better scalability than xGGEV3. Figure 17 shows the scaling for both algorithms as the number of threads is increased. Lastly, the xGGEV4 algorithm is backwards stable which was numerically verified with backward errors shown in Figure 18.



(a) (b)

Figure 16: (a) Runtimes for the xGGEV3 and the xGGEV4 algorithms on pencils of size 8000×8000 with increasing number of eigenvalues at infinity. The xGGEV4 algorithm has lower runtimes in all cases. (b) The relative runtimes between the xGGEV3 and xGGEV4 algorithms. The relative runtime is defined as (time xGGEV3)/(time xGGEV4). xGGEV4 is about 4.5 times faster for sufficiently large matrices.

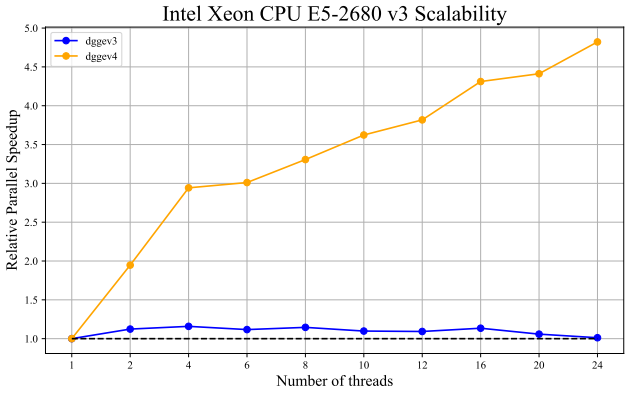


Figure 17: Comparison of the resulting backward errors for the computed left and right eigenvectors for random matrix pencils of varying size. The eigen decomposition computed using Algorithm 3 is not only faster, but numerically stable.

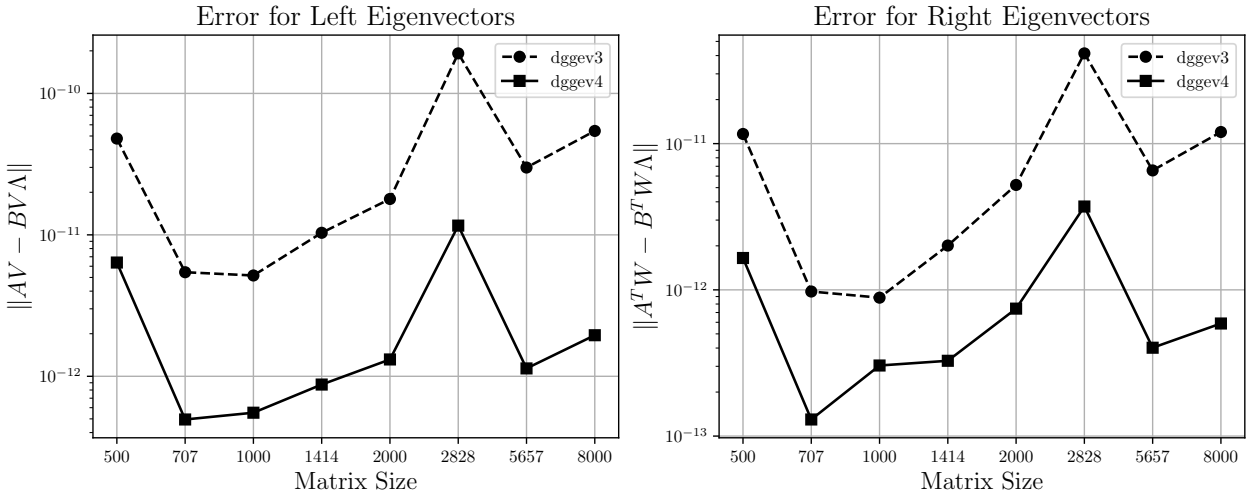


Figure 18: Comparison of the resulting backward errors for the computed left and right eigenvectors for random matrix pencils of varying size. The eigen decomposition computed using Algorithm 3 is not only faster, but numerically stable.

Algorithm 3 xGGEV4: generalized eigendecomposition of a matrix pencil $(A, B) \in \mathbb{R}^{n\times n}$

Input: $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times n}$
Output: Eigenvectors $Q \in \mathbb{R}^{n\times n}$, $Z \in \mathbb{R}^{n\times n}$ with reduced A and B

Step 1: Preprocess

▶ Perform RRRQ on B to move any infinite eigenvalue to leftmost columns for immediate deflation
J = permutation matrix to reverse columns when applied from right
$Q_B, B, P = \text{rrqr}(B^T J)$
$A = JP^T JAQ_B J$
$B = JB^T J$
$n_\infty = n - \text{rank}(B)$
$Q = JPJ$
$Z = Q_B J$

Step 2: Deflate All Infinite Eigenvalues

▶ If B was rank deficient, deflate the corresponding infinite eigenvalues
if $n_\infty > 0$ **then**
  $Q_A, A_{:,:n_\infty} = \text{qr}(A_{:,:n_\infty})$
  $A_{:,n_\infty:} = Q_A^T A_{:,n_\infty:}$
  $B_{:,n_\infty:} = Q_A^T B_{:,n_\infty:}$
  $Q = Q_A Q$
  $Q_B, B_{n_\infty:,n_\infty:} = \text{rq}(B_{n_\infty:,n_\infty:})$
  $A_{:,n_\infty:} = A_{:,n_\infty:} Q_B^T$
  $B_{:n_\infty,n_\infty:} = A_{:n_\infty,n_\infty:} Q_B^T$
  $Z_{:,n_\infty:} = Z_{:,n_\infty:} Q_B^T$
end if
$k = n_\infty$

Step 3: Perform Remaining Hessenberg-Triangular Reduction

▶ Use the algorithm from [17] to perfrom the rest of the reduction. $B_{k:,k:}$ will be full rank after preprocessing + deflation.
while $k < n$ **do**
  $X = A_{k:,k:} B_{k:,k:}^{-1}$
  $H_X, Q_c = \text{hessenberg}(X)$
  $A_{k:} = Q_c^T A_{k:}$
  $B_{k:} = Q_c^T B_{k:}$
  $Q_{:,k:} = Q_{:,k:} Q_c$
  $B_{k:,k:}, Zc = \text{rq}(B_{k:,k:})$
  $A_{:,k:} = A_{:,k:} Z_c^T$
  $B_{:k,k:} = B_{:k,k:} Z_c^T$
  $Z_{:,k:} = Z_{:,k:} Z_c^T$
  while $\|A_{k+2:,k}\| \le \varepsilon \|A\|$ **do**
    $k = k + 1$
  end while
end while

Step 4: Perform QZ Iteration on Reduced Pencil

xLAQZ0(A, B, Q, Z)
return A, B, Q, Z

Step 5: Generate Eigenvectors

xTGEVC(A, B, Q, Z)
Normalize eigenvectors to have unitary norm
return A, B, Q, Z

References

[1] E. Anderson et al. *LAPACK Users' Guide*. Third. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1999. ISBN: 0-89871-447-8 (paperback).

[2] Uri M Ascher, Robert M M Mattheij, and Robert D Russell. *Numerical solution of boundary value problems for ordinary differential equations*. Society for Industrial and Applied Mathematics, Jan. 1995.

[3] Carl de Boor and Blair Swartz. "Collocation at Gaussian Points". In: *SIAM Journal on Numerical Analysis* 10.4 (1973), pp. 582–606. DOI: 10.1137/0710052.

[4] G.F. Carey and Bruce Finlayson. "Orthogonal Collocation on Finite Elements". In: *Chemical Engineering Science* 30 (May 1975), pp. 587–596. DOI: 10.1016/0009-2509(75)80031-5.

[5] Timothy A. Davis. "Algorithm 832: UMFPACK V4.3—an unsymmetric-pattern multifrontal method". In: *ACM Trans. Math. Softw.* 30.2 (June 2004), pp. 196–199. ISSN: 0098-3500. DOI: 10.1145/992200.992206.

[6] Peter Deuflhard. *Newton methods for nonlinear problems*. en. 2004th ed. Springer series in computational mathematics. New York, NY: Springer, Jan. 2011.

[7] Eusebius J Doedel. "Nonlinear numerics". en. In: *J. Franklin Inst.* 334.5-6 (Sept. 1997), pp. 1049–1073.

[8] Leslie V. Foster. "The growth factor and efficiency of Gaussian elimination with rook pivoting". In: *Journal of Computational and Applied Mathematics* 86.1 (1997). Dedicated to William B. Gragg on the ocassion of his 60th Birthday, pp. 177–194. ISSN: 0377-0427. DOI: https://doi.org/10.1016/S0377-0427(97)00154-4.

[9] Carl Friedrich Gauss. *Methodus nova integralium valores per approximationem inveniendi*. 1814.

[10] Gene H Golub and Charles F Van Loan. *Matrix Computations*. en. 4th ed. Johns Hopkins Studies in the Mathematical Sciences. Baltimore, MD: Johns Hopkins University Press, Feb. 2013.

[11] Bo Kågström et al. "Blocked Algorithms for the Reduction to Hessenberg-Triangular Form Revisited LAPACK Working Note 198". In: (Mar. 2008).

[12] Yuri A Kuznetsov. *Elements of applied bifurcation theory*. en. 4th ed. Applied Mathematical Sciences. Cham, Switzerland: Springer International Publishing, Apr. 2023.

[13] H.P McKean. "Nagumo's equation". In: *Advances in Mathematics* 4.3 (1970), pp. 209–223. ISSN: 0001-8708. DOI: https://doi.org/10.1016/0001-8708(70)90023-X.

[14] C. B. Moler and G. W. Stewart. "An Algorithm for Generalized Matrix Eigenvalue Problems". In: *SIAM Journal on Numerical Analysis* 10.2 (1973), pp. 241–256. DOI: 10.1137/0710024.

[15] George Poole and Larry Neal. "The Rook's pivoting strategy". In: *Journal of Computational and Applied Mathematics* 123.1 (2000). Numerical Analysis 2000. Vol. III: Linear Algebra, pp. 353–369. ISSN: 0377-0427. DOI: https://doi.org/10.1016/S0377-0427(00)00406-4.

[16] Jens D.M. Rademacher, Björn Sandstede, and Arnd Scheel. "Computing absolute and essential spectra using continuation". In: *Physica D: Nonlinear Phenomena* 229.2 (2007), pp. 166–183. ISSN: 0167-2789. DOI: https://doi.org/10.1016/j.physd.2007.03.016.

[17] Thijs Steel and Raf Vandebril. "A novel, blocked algorithm for the reduction to Hessenberg-triangular form". In: *The Electronic Journal of Linear Algebra* (Oct. 2022), pp. 712–728. DOI: 10.13001/ela.2022.6483.

[18] Qian Wang et al. "AUGEM: Automatically generate high performance Dense Linear Algebra kernels on x86 CPUs". In: *SC '13: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 2013, pp. 1–12. DOI: 10.1145/2503210.2503219.